How-To Library

Informatica

Writing Custom Scripts to Execute Batch Jobs

© Copyright Informatica LLC 2013, 2021. Informatica Corporation. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica Corporation. All other company and product names may be trade names or trademarks of their respective owners and/or copyrighted materials of such owners.

Abstract

You can create custom scripts to run batch jobs and batch groups in an Informatica MDM Hub implementation. You can use the Batch Viewer and Batch Group tools to configure and run Informatica MDM Hub batch jobs. This article is intended for implementation teams and system administrators.

Supported Versions

- MDM Multidomain Edition for Oracle 9.5.1

Table of Contents

About Running Informatica MDM Hub Batch Jobs. . . . 3
Setting Up Scripts to Run Jobs. . . . 3
About Scripts to Run Jobs. . . . 4
About the C_REPOS_TABLE_OBJECT_V View. . . . 4
Determining Available Execution Scripts. . . . 7
Retrieving Values from C_REPOS_TABLE_OBJECT_V at Execution Time. . . . 7
Running Scripts Asynchronously. . . . 7
Monitoring Job Results and Statistics. . . . 7
Error Messages and Return Codes. . . . 8
Error Handling and Transaction Management. . . . 8
Housekeeping for Temporary Tables. . . . 8
Job Execution Status. . . . 9
Stored Procedure Reference. . . . 9
Alphabetical List of Batch Jobs. . . . 10
Accept Non-matched Records As Unique. . . . 12
Autolink Jobs. . . . 13
Auto Match and Merge Jobs. . . . 13
Automerge Jobs. . . . 14
Batch Unmerge Jobs. . . . 15
BVT Snapshot Jobs. . . . 18
Execute Batch Group Jobs. . . . 18
External Match Jobs. . . . 18
Generate Match Token Jobs. . . . 19
Get Batch Group Status Jobs. . . . 20
Hub Delete Jobs. . . . 20
Key Match Jobs. . . . 24
Load Jobs. . . . 25
Manual Link Jobs. . . . 26
Manual Merge Jobs. . . . 26
Manual Unlink Jobs. . . . 27

Manual Unmerge Jobs. . . . 27
Match Jobs. . . . 29
Match Analyze Jobs. . . . 31
Match for Duplicate Data Jobs. . . . 32
Multi Merge Jobs. . . . 32
Promote Jobs. . . . 34
Recalculate Base Object Jobs. . . . 34
Recalculate BVT Jobs. . . . 35
Reset Batch Group Status Jobs. . . . 36
Reset Links Jobs. . . . 36
Reset Match Table Jobs. . . . 36
Revalidate Jobs. . . . 37
Stage Jobs. . . . 37
Synchronize Jobs. . . . 38
Executing Batch Groups Using Stored Procedures. . . . 39
About Running Batch Groups. . . . 39
Stored Procedures for Batch Groups. . . . 40
CMXBG.EXECUTE_BATCHGROUP. . . . 40
CMXBG.RESET_BATCHGROUP. . . . 41
CMXBG.GET_BATCHGROUP_STATUS. . . . 42
Developing Custom Stored Procedures for Batch Jobs. . . . 44
About Custom Stored Procedures. . . . 44
Required Execution Parameters for Custom Batch Jobs. . . . 44
Registering a Custom Stored Procedure. . . . 45
Removing Data from a Base Object and Supporting Metadata Tables. . . . 46
Writing Messages to the Informatica MDM Hub Database Debug Log. . . . 46
Example Custom Stored Procedure. . . . 46

About Running Informatica MDM Hub Batch Jobs

An Informatica MDM Hub *batch job* is a program that, when run, completes a discrete unit of work (a process). All public batch jobs in Informatica MDM Hub can be run as database stored procedures.

In the Hub Console, the Informatica MDM Hub Batch Viewer and Batch Group tools provide simple mechanisms for executing Informatica MDM Hub batch jobs. However, they do not provide a means for executing and managing jobs on a scheduled basis. To run and manage jobs according to a schedule, you need to run stored procedures that do the work of batch jobs or batch groups. Most organizations have job management tools that are used to control IT processes. Any such tool capable of executing Oracle PL*SQL or DB2 SQL commands can be used to schedule and manage Informatica MDM Hub batch jobs.

Setting Up Scripts to Run Jobs

You need to set up scripts to run Informatica MDM Hub stored procedures.

About Scripts to Run Jobs

You can use scripts to run stored procedures on a scheduled basis to run and manage jobs.

Use scripts to perform the following tasks:

- Determine whether stored procedures can be run using job scheduling tools.
- Retrieve identifiers for scripts that run stored procedures.
- Determine which batch jobs are available to be run using stored procedures.
- Schedule stored procedures to run synchronously or asynchronously.

Informatica MDM Hub provides information regarding stored procedures, such as whether a stored procedure can be run using job scheduling tools, or how to retrieve identifiers that run stored procedures in the C_REPOS_TABLE_OBJECT_V view.

About the C_REPOS_TABLE_OBJECT_V View

The C_REPOS_TABLE_OBJECT_V view contains metadata and identifiers for the Informatica MDM Hub stored procedures.

Metadata in the C_REPOS_TABLE_OBJECT_V View

Informatica MDM Hub populates the C_REPOS_TABLE_OBJECT_V view with metadata about its stored procedures. Use this metadata to perform the following tasks:

- Determine whether a stored procedure can be run using job scheduling tools.
- Retrieve identifiers in the job execution scripts that execute Informatica MDM Hub stored procedures.

The following table describes the C_REPOS_TABLE_OBJECT_V columns:

Column Name	Description
ROWID_TABLE_OBJECT	Uniquely identifies a batch job.
ROWID_TABLE	Depending on the type of batch job, this is the table identifier for either the table affected by the job (target table) or the table providing the data for the job (source table). For Stage jobs, ROWID_TABLE refers to the target table (staging table). For Load jobs, ROWID_TABLE refers to the source table (staging table). For Match, Match Analyze, Autolink, Automerge, Auto Match and Merge, External Match, Generate Match Tokens, and Key Match jobs, ROWID_TABLE refers to the base object table, which is both source and target for the jobs.
OBJECT_NAME	Description of the type of batch job. The following examples provide a description of the type of batch job: Stage jobs: CMX_CLEANSE.EXE. Load jobs: CMXLD.LOAD_MASTER. Match and Match Analyze jobs: CMXMA.MATCH.

Column Name	Description
OBJECT_DESC	Description of the batch job, including the type of batch job as well as the object affected by the batch job. The following examples include a description of the object affected by the batch job: Stage for C_STG_CUSTOMER_CREDIT Load from C_STG_CUSTOMER_CREDIT Match and Merge for C_CUSTOMER
OBJECT_TYPE_CODE	Together with OBJECT_FUNCTION_TYPE_CODE, this is a foreign key to C_REPOS_OBJ_FUNCTION_TYPE. An OBJECT_TYPE_CODE of "P" indicates a procedure that can potentially be executed by a scheduling tool.
OBJECT_FUNCTION_TYPE_CODE	Indicates the actual procedure type such as stage, load, and match.
PUBLIC_IND	Indicates whether the procedure is a procedure that can be displayed in the Batch Viewer.
PARAMETER	Describes the parameter list for the procedure. Where specific ROWID_TABLE values are required for the procedure, these are shown in the parameter list. Otherwise, the name of the parameter is displayed in the parameter list. An exception to this is the parameter list for Stage jobs (where OBJECT_NAME = CMX_CLEANSE.EXE). In this case, the full parameter list is not shown.
VALID_IND	If VALID_IND is not equal to 1, do not execute the procedure. It means that some repository settings have changed that affect the procedure. This usually applies to changes that affect the Stage jobs if the mappings are not checked and saved again.

Identifiers in the C_REPOS_TABLE_OBJECT_V View

Use the following identifier values in C_REPOS_TABLE_OBJECT_V to run stored procedures:

Object Name	Object Description	Object Type Code	Object Function Type Code	Object Function Type Description
CMXUT.ACCEPT_NON_MATCH_UNIQUE	Change the status of records that have undergone the match process but had no matching data.	P	U	Accept Non-matched Records As Unique
CMXMM.AUTOLINK	Link data in *BaseObjectName*	P (Procedure)	I	Autolink
CMXMM.AUTOMERGE	Merge data in *BaseObjectName*	P (Procedure)	G	Automerge
CMXMM.BUILD_BVT	Generate BVT snapshot for *BaseObjectName*	P	V	BVT snapshot
CMXMA.EXTERNAL_MATCH	External Match for *BaseObjectName*	P	E	External match

Object Name	Object Description	Object Type Code	Object Function Type Code	Object Function Type Description
CMXMA.GENERATE_MATCH_TOKENS	Generate Match Tokens for *BaseObjectName*	P	N	Generate match tokens
CMXMA.KEY_MATCH	Key Match for *BaseObjectName*	P	K	Key match
CMXLD.LOAD_MASTER	Load from Link *BaseObjectName*	P	L	Load
CMXMM.MERGE	Process records that have been queued by a Match job for manual merge.	P	Y	Manual merge
CMXMA.MATCH	Match Analyze for *BaseObjectName*	P	Z	Match analyze
CMXMA.MATCH	Match for *BaseObjectName*	P	M	Match
CMXMA.MATCH_AND_MERGE	Match and Merge for *BaseObjectName*	P	B	Auto match and merge
CMXMA.MATCH_FOR_DUPS	Match for Duplicate Data for *BaseObjectName*	P	D	Match for duplicate data
CMXMM.MLINK	Manual Link for *BaseObjectName*	P	O	Manual link
CMXMA.MIGRATE_LINK_STYLE_TO_MERGE_STYLE	Migrate Link Style to Merge Style for *BaseObjectName*	P	J	Migrate link style to merge style
CMXMM.MULTI_MERGE	Multi Merge for *BaseObjectName*	P	P	Multi merge
CMXSM.AUTO_PROMOTE	Reads the PROMOTE_IND column from an XREF table and for all rows where the column's value is 1, changes the ACTIVE state to on.	P	PR	Promote
CMXMM.MUNLINK	Manual Unlink for *BaseObjectName*	P	Q	Manual unlink
CMXMA.RESET_LINKS	Reset Links for *BaseObjectName*	P	W	Reset links
CMXMA.RESET_MATCH	Reset Match table for *BaseObjectName*	P	R	Reset match table

Object Name	Object Description	Object Type Code	Object Function Type Code	Object Function Type Description
CMXUT.REVALIDATE_BO	Revalidate *BaseObjectName*	P	H	Revalidate BO
CMXCL.START_CLEANSE	Stage for *TargetStagingTableName*	P	C	Stage
CMXUT.SYNC	Synchronize after changes are made to the schema trust settings.	P	S	Synchronize
CMXMM.UNMERGE	Unmerge for *BaseObjectName*	P	X	Manual unmerge

Determining Available Execution Scripts

To determine which batch jobs are available to be executed using stored procedures, run a query using the standard Informatica MDM Hub view called C_REPOS_TABLE_OBJECT_V:

```
SELECT *
FROM C_REPOS_TABLE_OBJECT_V
WHERE  PUBLIC_IND = 1 :
```

Retrieving Values from C_REPOS_TABLE_OBJECT_V at Execution Time

Use SQL statements to retrieve values from C_REPOS_TABLE_OBJECT_V when you run scripts at run time. The following example code retrieves the STG_ROWID_TABLE and ROWID_TABLE_OBJECT for cleanse jobs:

```
SELECT A.ROWID_TABLE, A.ROWID_TABLE_OBJECT INTO IN_STG_ROWID_TABLE, IN_ROWID_TABLE_OBJECT
FROM C_REPOS_TABLE_OBJECT_V A, C_REPOSE_TABLE B
WHERE A.OBJECT_NAME = 'CMX_CLEANSE.EXE'
AND B.ROWID_TABLE = A.ROWID_TABLE
AND B.TABLE_NAME = 'C_HMO_ADDRESS'
AND A.VALID_IND = 1;
```

Running Scripts Asynchronously

By default, the execution scripts run synchronously (IN_RUN_SYNCH = ‘TRUE’ or IN_RUN_SYNCH = NULL). To run the execution scripts asynchronously, specify IN_RUN_SYNCH = ‘FALSE’. Note that these Boolean values are case sensitive and must be specified in uppercase characters.

Monitoring Job Results and Statistics

You can monitor the results and view the associated statistics of batch jobs run in job execution scripts.

Error Messages and Return Codes

Informatica MDM Hub stored procedures return an error message and return code. The following table describes the return codes:

Returned Parameter	Description
OUT_ERROR_MSG	Error message if an error occurs.
OUT_RETURN_CODE	Return code. The return code is `0` if no errors occurs, or `1` if an error occurs.

Error handling code in job execution scripts can look for return codes and trap any associated error messages.

Error Handling and Transaction Management

The stored procedures are transaction-enabled and can be rolled back if an error occurs during execution.

After you invoke a stored procedure, check the return code (OUT_RETURN_CODE) in your error handling:

- If any failure occurred during execution (OUT_RETURN_CODE <> 0), *immediately* roll back any changes. Wait until *after* you have successfully rolled back the changes before you invoke the stored procedure again.
- If no failure occurred during execution (OUT_RETURN_CODE = 0), *commit* any changes.

Housekeeping for Temporary Tables

Informatica MDM Hub stored procedures, when invoked directly, generally clean up any internal temporary files created during execution. However, in the following cases, certain stored procedures do not clean up internal temporary files created during execution:

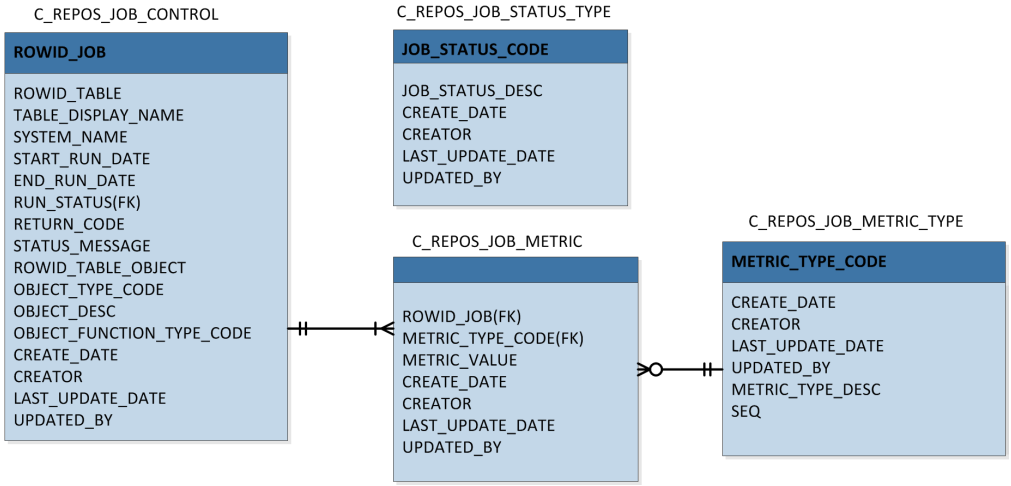
- Certain stored procedures have an OUT_TMP_TABLE_LIST return parameter, which consists of temporary tables that contain data that could be useful for debugging purposes. For such stored procedures, if OUT_RETURN_CODE=0 is returned, pass the returned OUT_TMP_TABLE_LIST parameter to the CMXUT.DROP_TEMP_TABLES stored procedure to clean up the temporary tables that were returned in the parameter.

```
IF rc = 0 THEN
   COMMIT;
   cmxut.drop_table_in_list( out_tmp_table_list, out_error_message, rc );
END IF;
```

- Certain stored procedures will also register the temporary tables that remain so that a server-side process can periodically remove them.

Job Execution Status

Informatica MDM Hub stored procedures log their job execution status and statistics in the Informatica MDM Hub repository. The following figure shows the repository tables that can be used for monitoring job results and statistics:



The following table describes the various repository tables used for monitoring job results and statistics:

Table Name	Description
C_REPOS_JOB_CONTROL	As soon as a job starts to run, it registers itself in C_REPOS_JOB_CONTROL with a RUN_STATUS of 2 (Running/Processing). Once the job completes, its status is updated to one of the following values: - **0 (Completed Successfully)**—Completed without any errors or warnings. - **1 (Completed with Errors)**—Completed, but with some warnings or data rejections. See the RETURN_CODE for any error code and the STATUS_MESSAGE for a description of the error/warning. - **2 (Running / Processing)** - **3 (Failed—Job did not complete)**. Corrective action must be taken and the job must be run again. See the RETURN_CODE for any error code and the STATUS_MESSAGE for the reason for failure. - **4 (Incomplete)**—The job failed before updating its job status and has been manually marked as incomplete. Corrective action must be taken and the job must be run again. RETURN_CODE and STATUS_MESSAGE will not provide any useful information. Marked as incomplete by clicking the Set Status to Incomplete button in the Batch Viewer.
C_REPOS_JOB_METRIC	When a batch job has completed, it registers its statistics in C_REPOS_JOB_METRIC. There can be multiple statistics for each job. Join to C_REPOS_JOB_METRIC_TYPE to get a description for each statistic.
C_REPOS_JOB_METRIC_TYPE	Stores the descriptions of the types of metrics that can be registered in C_REPOS_JOB_METRIC.
C_REPOS_JOB_STATUS_TYPE	Stores the descriptions of the RUN_STATUS values that can be registered in C_REPOS_JOB_CONTROL.

Stored Procedure Reference

This section provides a reference for the stored procedures that represent Informatica MDM Hub batch jobs. Informatica MDM Hub provides these stored procedures, in compiled form, for each Operational Reference Store (ORS), for Oracle databases. You can use any job scheduling software (such as Tivoli, CA Unicenter, and so on) to execute these stored procedures.

Note: All the input parameters that need a delimited list require a trailing "~" character.

Alphabetical List of Batch Jobs

Batch Job	Description	Reference
Accept Non-Matched Records As Unique	For records that have undergone the match process but had no matching data, sets the consolidation indicator to 1 (consolidated), meaning that the record was unique and did not require consolidation.	"Accept Non-matched Records As Unique" on page 12
Autolink Jobs	Automatically links records that have qualified for autolinking during the match process and are flagged for autolinking (Autolink_ind=1). Used with link-style base objects only.	"Autolink Jobs" on page 13
Auto Match and Merge Jobs	Executes a continual cycle of a Match job, followed by an Automerge job, until there are no more records to match, or until the size of the manual merge queue exceeds the configured threshold. Used with merge-style base objects only.	"Auto Match and Merge Jobs" on page 13
Automerge Jobs	Automatically merges records that have qualified for automerging during the match process and are flagged for automerging (Automerge_ind=1). Used with merge-style base objects only.	"Automerge Jobs" on page 14
Batch Unmerge Jobs	Unmerges records consolidated using Automerge, Manual Merge, manual edit, Load by ROWID_OBJECT, or Put XREF.	"Batch Unmerge Jobs" on page 15
BVT Snapshot Jobs	Generates a snapshot of the best version of the truth (BVT) for a base object. Used with link-style base objects only.	"BVT Snapshot Jobs" on page 18
Execute Batch Group Jobs	Constructs an XML message and sends it to the MRM Server SIF API (ExecuteBatchGroupRequest), which performs the operation.	"Execute Batch Group Jobs" on page 18
External Match Jobs	Matches "externally managed/prepared" records with an existing base object, yielding the results based on the current match settings—all without actually modifying the data in the base object.	"External Match Jobs" on page 18
Generate Match Token Jobs	Prepares data for matching by generating match tokens according to the current match settings. Match tokens are strings that encode the columns used to identify candidates for matching.	"Generate Match Token Jobs" on page 19
Get Batch Group Status Jobs	Returns the status of a batch group.	"Get Batch Group Status Jobs" on page 20
Hub Delete Jobs	Deletes data from the Hub based on base object / XREF level input.	"Hub Delete Jobs" on page 20
Key Match Jobs	Matches records from two or more sources when these sources use the same primary key. Compares new records to each other and to existing records, and identifies potential matches based on the comparison of source record keys as defined by the match rules.	"Key Match Jobs" on page 24
Load Jobs	Copies records from a staging table to the corresponding target base object in the Hub Store. During the load process, it also applies the current trust and validation rules to the records.	"Load Jobs" on page 25

Batch Job	Description	Reference
Manual Link Jobs	Shows logs for records that have been manually linked in the Merge Manager tool. Used with link-style base objects only.	"Manual Link Jobs" on page 26
Manual Merge Jobs	Shows logs for records that have been manually merged in the Merge Manager tool. Used with merge-style base objects only.	"Manual Merge Jobs" on page 26
Manual Unlink Jobs	Shows logs for records that have been manually unlinked in the Data Manager tool. Used with link-style base objects only.	"Manual Unlink Jobs" on page 27
Manual Unmerge Jobs	Shows logs for records that have been manually unmerged in the Merge Manager tool. Used with merge-style base objects only.	"Manual Unmerge Jobs" on page 27
Match Jobs	Finds duplicate records in the base object, based on the current match rules.	"Match Jobs" on page 29
Match Analyze Jobs	Conducts a search to gather match statistics but does not actually perform the match process. If areas of data with the potential for huge match requirements are discovered, Informatica MDM Hub moves the records to a hold status, which allows a data steward to review the data manually before proceeding with the match process.	"Match Analyze Jobs" on page 31
Match for Duplicate Data Jobs	For data with a high percentage of duplicate records, compares new records to each other and to existing records, and identifies exact duplicates. The maximum number of exact duplicates is based on the Duplicate Match Threshold setting for this base object. **Note:** The Match for Duplicate Data batch job has been deprecated.	"Match for Duplicate Data Jobs" on page 32
Promote Jobs	Reads the PROMOTE_IND column from an XREF table and changes to ACTIVE the state on all rows where the column's value is 1.	"Promote Jobs" on page 34
Recalculate Base Object Jobs	Recalculates all base objects or base objects you specify with the ROWID_OBJECT_TABLE parameter.	Recalculate BO Jobs
Recalculate BVT Jobs	Recalculates the BVT for the specified ROWID_OBJECT.	Recalculate BVT Jobs
Reset Batch Group Status Jobs	Resets a batch group.	"Reset Batch Group Status Jobs" on page 36
Reset Links Jobs	Updates the records in the _LINK table to account for changes in the data. Used with link-style base objects only.	"Reset Links Jobs" on page 36
Reset Match Table Jobs	Shows logs of the operation where all matched records have been reset to be queued for match.	"Reset Match Table Jobs" on page 36
Revalidate Jobs	Executes the validation logic/rules for records that have been modified since the initial validation during the Load Process. You can run Revalidate if/when records change after the initial Load process's validation step. If no records change, no records are updated. If some records have changed and get caught by the existing validation rules, the metrics will show the results.	"Revalidate Jobs" on page 37

Batch Job	Description	Reference
Stage Jobs	Copies records from a landing table into a staging table. During execution, cleanses the data according to the current cleanse settings.	"Stage Jobs" on page 37
Synchronize Jobs	Updates metadata for base objects. Used after a base object has been loaded but not yet merged, and subsequent trust configuration changes (such as enabling trust) have been made to columns in that base object. This job must be run before merging data for this base object.	"Synchronize Jobs" on page 38

Accept Non-matched Records As Unique

The Accept Non-Matched Records As Unique batch job changes the status of records without matches. This batch job sets the consolidation indicator to "1" for the records without any matches. A value of "1" indicates that the MDM Hub does not need to consolidate the record. The Manual Merge operation also sets the consolidation indicator to "1" for the target record if it does not have any matches. The Automerge batch job considers records with a consolidation indicator of "1" as unique records.

The MDM Hub creates the Accept Non-Matched Records As Unique batch job after a Merge batch job if Accept All Unmatched Rows as Unique is enabled.

Note: You cannot run the Accept Non-Matched Records As Unique batch job from the Batch Viewer.

Stored Procedure Definition for Accept Non-matched Records As Unique Jobs

```
PROCEDURE CMXUT.ACCEPT_NON_MATCH_UNIQUE (
 IN_ROWID_TABLE             IN CHAR(14)
,IN_ROWID_USER              IN CHAR(14)
,IN_ASSIGNMENT_IND          INT
,OUT_ACCEPT_UNIQUE_CNT      OUT INT
,OUT_ERROR_MSG              OUT VARCHAR2(1024)
,RC                         OUT INT
,OUT_TMP_TABLE_LIST         OUT CLOB
,IN_ROWID_JOB               CMXLB.CMX_ROWID DEFAULT NULL);
```

Sample Job Execution Script for Accept Non-matched Records As Unique

```
-- ACCEPT RECORDS ASSIGNED TO ALL USERS
DECLARE
    V_ROWID_TABLE           CHAR( 14 );
    OUT_ACCEPT_UNIQUE_CNT   INTEGER;
    OUT_ERROR_MESSAGE       VARCHAR2( 1024 );
    OUT_RETURN_CODE         INTEGER;
    OUT_TMP_TABLE_LIST      VARCHAR2(32000);
BEGIN
    SELECT ROWID_TABLE
    INTO   V_ROWID_TABLE
    FROM   C_REPOS_TABLE
    WHERE  TABLE_NAME = 'C_CUSTOMER';

    CMXUT.ACCEPT_NON_MATCH_UNIQUE( V_ROWID_TABLE, NULL, 0, OUT_ACCEPT_UNIQUE_CNT,
OUT_ERROR_MESSAGE, OUT_RETURN_CODE, OUT_TMP_TABLE_LIST );
    DBMS_OUTPUT.PUT_LINE( 'NUMBER FOR RECORDS ACCEPTED AS UNIQUE: ' || OUT_ACCEPT_UNIQUE_CNT );
    DBMS_OUTPUT.PUT_LINE( 'RETURN MESSAGE: ' || SUBSTR( OUT_ERROR_MESSAGE, 1, 255 ));
    DBMS_OUTPUT.PUT_LINE( 'RETURN CODE: ' || OUT_RETURN_CODE );

    IF OUT_RETURN_CODE<0 THEN
       ROLLBACK;
    ELSE
       COMMIT;
    END IF;
    CMXUT.DROP_TABLE_IN_LIST(OUT_TMP_TABLE_LIST,OUT_ERROR_MESSAGE,OUT_RETURN_CODE); --Clean up
```

```
temporary tables.
END;
/
-- ACCEPT ONLY RECORDS ASSIGNED TO SPECIFIC USER
DECLARE
    V_ROWID_TABLE           CHAR( 14 );
    V_ROWID_USER            CHAR( 14 );
    OUT_ACCEPT_UNIQUE_CNT   INTEGER;
    OUT_ERROR_MESSAGE       VARCHAR2( 1024 );
    OUT_RETURN_CODE         INTEGER;
    OUT_TMP_TABLE_LIST      VARCHAR2(32000);
BEGIN
    SELECT ROWID_TABLE
    INTO   V_ROWID_TABLE
    FROM   C_REPOS_TABLE
    WHERE  TABLE_NAME = 'C_CUSTOMER';

    SELECT ROWID_USER
    INTO   V_ROWID_USER
    FROM   C_REPOS_USER
    WHERE  UPPER(USER_NAME) = 'ADMIN';

    CMXUT.ACCEPT_NON_MATCH_UNIQUE( V_ROWID_TABLE, V_ROWID_USER, 1, OUT_ACCEPT_UNIQUE_CNT,
OUT_ERROR_MESSAGE, OUT_RETURN_CODE, OUT_TMP_TABLE_LIST );
    DBMS_OUTPUT.PUT_LINE( 'NUMBER FOR RECORDS ACCEPTED AS UNIQUE: ' || OUT_ACCEPT_UNIQUE_CNT );
    DBMS_OUTPUT.PUT_LINE( 'RETURN MESSAGE: ' || SUBSTR( OUT_ERROR_MESSAGE, 1, 255 ));
    DBMS_OUTPUT.PUT_LINE( 'RETURN CODE: ' || OUT_RETURN_CODE );

    IF OUT_RETURN_CODE<0 THEN
        ROLLBACK;
    ELSE
        COMMIT;
    END IF;
    CMXUT.DROP_TABLE_IN_LIST(OUT_TMP_TABLE_LIST,OUT_ERROR_MESSAGE,OUT_RETURN_CODE); --Clean up
temporary tables.
END;
/
```

Autolink Jobs

Autolink jobs automatically link records that have qualified for autolinking during the match process and are flagged for autolinking (Autolink_ind = 1).

Auto Match and Merge Jobs

Auto Match and Merge batch jobs execute a continual cycle of a Match job, followed by an Automerge job, until there are no more records to match, or until the size of the manual merge queue exceeds the configured threshold. Auto Match and Merge jobs are used with merge-style base objects only.

Note: Do not run an Auto Match and Merge job on a base object that is used to define relationships between records in inter-table or intra-table match paths. Doing so will change the relationship data, resulting in the loss of the associations between records.

Identifiers for Executing Auto Match and Merge Jobs

Identifiers are used to execute the stored procedure associated with this batch job.

Dependencies for Auto Match and Merge Jobs

The Auto Match and Merge jobs for a target base object can either be run on successful completion of each Load job, or on successful completion of all Load jobs for the object.

Successful Completion of Auto Match and Merge Jobs

Auto Match and Merge jobs must complete with a RUN_STATUS of 0 (Completed Successfully) or 1 (Completed with Errors) to be considered successful.

Stored Procedure Definition for Auto Match and Merge Jobs

```
PROCEDURE CMXMA.MATCH_AND_MERGE (
 IN_ROWID_TABLE             IN CHAR(14) --Rowid of a table
,IN_USER_NAME               IN VARCHAR2(50) --User name
,IN_MATCH_SET_NAME          IN VARCHAR2(500) DEFAULT NULL
,OUT_ERROR_MSG              OUT VARCHAR2(1024) --Error message, if any
,RC                         OUT INT
,IN_JOB_GRP_CTRL            IN CHAR(14) DEFAULT NULL
,IN_JOB_GRP_ITEM            IN CHAR(14) DEFAULT NULL
)
```

Sample Job Execution Script for Auto Match and Merge Jobs

```
DECLARE
   IN_ROWID_TABLE           CHAR(14);
   IN_USER_NAME             VARCHAR2(50);
   IN_MATCH_SET_NAME        VARCHAR(500);
   OUT_ERROR_MSG            VARCHAR2(1024);
   OUT_RETURN_CODE          NUMBER;
BEGIN
   IN_ROWID_TABLE           := 'SVR1.188';
   IN_USER_NAME             := 'CMX_ORS';
   IN_MATCH_SET_NAME        := 'MRS2';
   OUT_ERROR_MSG            := NULL;
   OUT_RETURN_CODE          := NULL;
   CMXMA.MATCH_AND_MERGE (IN_ROWID_TABLE, IN_USER_NAME, IN_MATCH_SET_NAME, OUT_ERROR_MSG,
OUT_RETURN_CODE);
   DBMS_OUTPUT.PUT_LINE ('OUT_ERROR_MESSAGE = ' || OUT_ERROR_MESSAGE);
   DBMS_OUTPUT.PUT_LINE ('RC = ' || TO_CHAR(OUT_RETURN_CODE));
   IF OUT_RETURN_CODE<0 THEN
        ROLLBACK;
   ELSE
        COMMIT;
   END IF;
END;
```

Automerge Jobs

Automerge jobs automatically merge records that have qualified for automerging during the match process and are flagged for automerging (Automerge_ind = 1). Automerge jobs are used with merge-style base objects only.

Identifiers for Executing Automerge Jobs

Identifiers are used to execute the stored procedure associated with this batch job.

Dependencies for Automerge Jobs

Each Automerge job is dependent on the successful completion of the match process, and the queuing of records for automerge.

Successful Completion of Automerge Jobs

Automerge jobs must complete with a RUN_STATUS of 0 (Completed Successfully) or 1 (Completed with Errors) to be considered successful.

Stored Procedure Definition for Automerge Jobs

```
PROCEDURE CMXMM.AUTOMERGE (
 IN_ROWID_TABLE             CMXLB.CMX_ROWID
,IN_USER_NAME               CMXLB.CMX_USER_NAME
,OUT_ERROR_MESSAGE          OUT CMXLB.CMX_MESSAGE
,OUT_RETURN_CODE            OUT INT
,IN_ROWID_JOB_GRP_CTRL      CMXLB.CMX_ROWID DEFAULT NULL
,IN_ROWID_JOB_GRP_ITEM      CMXLB.CMX_ROWID DEFAULT NULL
)
```

Sample Job Execution Script for Automerge Jobs

```
DECLARE
   IN_ROWID_TABLE           CHAR(14);
   IN_USER_NAME             VARCHAR2(50);
   OUT_ERROR_MESSAGE        VARCHAR2(1024);
   OUT_RETURN_CODE          NUMBER;
BEGIN
   IN_ROWID_TABLE           := NULL;
   IN_USER_NAME             := NULL;
   OUT_ERROR_MESSAGE        := NULL;
   OUT_RETURN_CODE          := NULL;
   CMXMM.AUTOMERGE (IN_ROWID_TABLE, IN_USER_NAME, OUT_ERROR_MESSAGE, OUT_RETURN_CODE);
   DBMS_OUTPUT.PUT_LINE ('OUT_ERROR_MESSAGE = ' || OUT_ERROR_MESSAGE);
   DBMS_OUTPUT.PUT_LINE ('OUT_RETURN_CODE = ' || TO_CHAR(OUT_RETURN_CODE));
   IF OUT_RETURN_CODE < 0 THEN
      ROLLBACK;
   ELSE
      COMMIT;
   END IF;
END;
```

Batch Unmerge Jobs

The Batch Unmerge job unmerges records consolidated using Automerge, Manual Merge, manual edit, Load by ROWID_OBJECT, or Put XREF. The Batch Unmerge job functions similar to the Manual Unmerge job with some differences. After a Batch Unmerge, edits are not removed from the remaining XREFs except in the case of unmerge with the EXPLODE_NODE_IND = 1 parameter. When EXPLODE_NODE_IND = 1, edits are removed but are included in the audit table of unmerged XREF records.

Batch Unmerge does not remove matches that are pointed to for the records that are unmerged in the parent or any related child tables. Use a calling procedure that uses the output list of processed records to remove matches.

Batch Unmerge jobs are dependent on data that is already merged. The batch_unmerge process accepts an input table, IN_XREF_LIST_TO_BE_UNMERGED, which contains a list of the source system records to be unmerged. The source system records can be identified either by the primary key, which is the ROWID_XREF column or the source system key, which is a combination of PKEY_SRC_OBJECT and ROWID_SYSTEM columns. If the input table has all 3 columns, then the ROWID_XREF column is used even if it is null. The input table also has an EXPLODE_NODE_IND column to indicate whether the source system records will unmerge the entire base object record.

You can create an input table using the following statement:

```
CREATE TABLE IN_XREF_LIST_TO_BE_UNMERGED
(
ROWID_XREF              INT,
PKEY_SRC_OBJECT    VARCHAR (255),
ROWID_SYSTEM        CHAR (14),
EXPLODE_NODE_IND    INT DEFAULT 0
)
```

Optionally, the Batch Unmerge uses an output table, OUT_UNMERGED_XREF_LIST, for audit purpose. The output table contains all the source system records that are unmerged, including records from all the cascading child tables.

You can create an output table using the following statement:

```
CREATE TABLE OUT_UNMERGED_XREF_LIST
(
  BO_TABLE              VARCHAR2(50),
  ROWID_XREF            INT,
  PKEY_SRC_OBJECT       VARCHAR2(255),
  ROWID_SYSTEM          CHAR(14),
  EXPLODE_NODE_IND      INT,
  PROCESSED_DATE        DATE
)
```

The Batch Unmerge job succeeds or fails as a single transaction. If the server fails when an unmerge job is running, the unmerge process is rolled back.

Note: The number of records in the OUT_UNMERGED_XREF_COUNT returned parameter does not include the records added by the EXPLODE_NODE_IND = 1 parameter.

Cascade Unmerge

The Batch Unmerge job can perform a cascade unmerge. You can enable cascade unmerge for a base object through the Schema Manager in the Hub Console. Cascade unmerge unmerges affected records in the child base object, when records in the parent object are unmerged. This feature applies to unmerging records across base objects.

You can configure cascade unmerge for each base object by modifying the merge settings in the Schema Manager to unmerge child when parent unmerges. Cascade unmerge applies only when a foreign-key relationship exists between two base objects.

For example, Customer A record (parent) in the Customer base object has multiple address records (children) in the Address base object and the two tables are linked by a unique key (Customer_ID). You can unmerge the parent record (Customer A) in the following scenarios:

When cascade unmerge is enabled.

Unmerging the parent record (Customer A) in the Customer base object also unmerges Customer A's child address records in the Address base object.

When cascade unmerge is disabled.

Unmerging the parent record (Customer A) in the Customer base object has no effect on Customer A's child records in the Address base object; they are NOT unmerged.

Linear Unmerge

During a linear unmerge, a base object record is unmerged and taken out of the existing merge tree structure. Only the unmerged base object record itself comes out of the merge tree structure, and all base object records below it in the merge tree stay in the original merge tree.

Merge tree structures that are built during the merge process are flattened after a Batch Unmerge process is executed. A tree unmerge is not available after a batch unmerge call.

Committing Unmerge Transactions After Error Checking

The Batch Unmerge stored procedure is transactional and can be rolled back if an error occurs during execution. If IN_ALLOW_COMMIT_IND = 1, Batch Unmerge executes COMMIT or ROLLBACK and cleans up all the temporary tables. If IN_ALLOW_COMMIT_IND = 0, Batch Unmerge does not execute COMMIT or ROLLBACK. After you call the Batch Unmerge job, check the return code, OUT_RETURN_CODE, in your error handler.

- If any failure occurs during execution (OUT_RETURN_CODE < 0), immediately roll back any changes. Wait until after you have successfully rolled back the unmerge changes before you invoke Unmerge again.
- If no failure occurs during execution (OUT_RETURN_CODE = 0), commit any unmerge changes.

Successful Completion of Batch Unmerge Jobs

A Batch Unmerge job completes with a RUN_STATUS of 0 if it completes successfully or with a RUN_STATUS of 1 if it completes with errors.

Note: You must execute the CMXUT.DROP_TABLE_IN_LIST stored procedure after the Batch Unmerge procedure to clean the OUT_TMP_TABLE_LIST temporary tables.

Stored Procedure Definition for Batch Unmerge Jobs

The following sample shows a stored procedure definition for a Batch Unmerge Job:

```
PROCEDURE BATCH_UNMERGE(
IN_BO_TABLE_NAME                IN    CMXLB.CMX_TABLE_NAME,
IN_XREF_LIST_TO_BE_UNMERGED     IN     CMXLB.CMX_TABLE_NAME,
OUT_UNMERGED_XREF_LIST             IN     CMXLB.CMX_TABLE_NAME,
IN_ROWID_JOB_CTL             IN     CMXLB.CMX_ROWID,
IN_INTERACTION_ID            IN    INT,
OUT_UNMERGED_XREF_COUNT             OUT    INT,
OUT_ERROR_MSG                   OUT    CMXLB.CMX_MESSAGE,
OUT_RETURN_CODE              OUT    INT,
OUT_TMP_TABLE_LIST           IN OUT CMXLB.CMX_BIG_STR,
IN_USER_NAME                 IN     CMXLB.CMX_USER_NAME DEFAULT NULL,
IN_ALLOW_COMMIT_IND          IN     INT DEFAULT 1
)
```

Sample Job Execution Script for Batch Unmerge Jobs

This is the start of your concept.

```
DECLARE
  OUT_ERROR_MESSAGE           VARCHAR2(1024);
  OUT_RETURN_CODE             NUMBER;
  OUT_TMP_TABLE_LIST          VARCHAR2(32000);
  OUT_UNMERGED_XREF_COUNT     NUMBER;
  IN_BO_TABLE_NAME            CMXLB.CMX_TABLE_NAME := 'C_UNMRG01';
  IN_XREF_LIST_TO_BE_UNMERGED CMXLB.CMX_TABLE_NAME := 'TEST_BATCH_UNMERGE';
  OUT_UNMERGED_XREF_LIST      CMXLB.CMX_TABLE_NAME := 'OUT_UNMERGED_XREF_LIST';
BEGIN
  BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE ' || IN_XREF_LIST_TO_BE_UNMERGED;
  EXCEPTION
    WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE(' SQLERRM = ' || SQLERRM);
  END;

  BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE ' || OUT_UNMERGED_XREF_LIST;
  EXCEPTION
    WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE(' SQLERRM = ' || SQLERRM);
  END;

  EXECUTE IMMEDIATE 'CREATE TABLE ' || OUT_UNMERGED_XREF_LIST || '('
                || 'BO_TABLE         VARCHAR2(50),'
                || 'ROWID_XREF       INTEGER,'
                || 'PKEY_SRC_OBJECT  VARCHAR2(255),'
                || 'ROWID_SYSTEM     CHAR(14),'
                || 'EXPLODE_NODE_IND INTEGER,'
                || 'PROCESSED_DATE   DATE)';

  EXECUTE IMMEDIATE 'CREATE TABLE ' || IN_XREF_LIST_TO_BE_UNMERGED || ' AS SELECT '
            --  || ' PKEY_SRC_OBJECT, ROWID_SYSTEM'  -- you can use PKEY_SRC_OBJECT +
ROWID_SYSTEM to identify XREFs to be unmerged
                || ' ROWID_XREF'                     -- or ROWID_XREF, if you specify all 3
columns, then only ROWID_XREF will be used
                || ', 0 EXPLODE_NODE_IND FROM ' || IN_BO_TABLE_NAME || '_XREF WHERE ROWNUM =
1';
```

```
  CMXUM.BATCH_UNMERGE(IN_BO_TABLE_NAME            => IN_BO_TABLE_NAME,
                      IN_XREF_LIST_TO_BE_UNMERGED => IN_XREF_LIST_TO_BE_UNMERGED,
                      OUT_UNMERGED_XREF_LIST      => OUT_UNMERGED_XREF_LIST,
                      IN_ROWID_JOB_CTL            => NULL,
                      IN_INTERACTION_ID           => NULL,
                      OUT_UNMERGED_XREF_COUNT     => OUT_UNMERGED_XREF_COUNT,
                      OUT_ERROR_MSG               => OUT_ERROR_MESSAGE,
                      OUT_RETURN_CODE             => OUT_RETURN_CODE,
                      OUT_TMP_TABLE_LIST          => OUT_TMP_TABLE_LIST,
                      IN_USER_NAME                => 'admin',
                      IN_ALLOW_COMMIT_IND         => 1);

  IF OUT_RETURN_CODE < 0 THEN
    ROLLBACK;
  ELSE
    COMMIT;
  END IF;

  DBMS_OUTPUT.PUT_LINE(' MESSAGE IS = ' || OUT_ERROR_MESSAGE);
  DBMS_OUTPUT.PUT_LINE(' RETURN CODE = ' || OUT_RETURN_CODE);
  DBMS_OUTPUT.PUT_LINE(' NUMBER OF UNMERGED XREFs = ' || OUT_UNMERGED_XREF_COUNT);

  CMXUT.DROP_TABLE_IN_LIST(OUT_TMP_TABLE_LIST,
                           OUT_ERROR_MESSAGE,
                           OUT_RETURN_CODE); --clean up temporary tables

END;
```

BVT Snapshot Jobs

The BVT Snapshot stored procedure generates a snapshot of the best version of the truth (BVT) for a base object.

Execute Batch Group Jobs

Execute Batch Group jobs (CMXBG.EXECUTE_BATCHGROUP) execute a batch group. Note that there are two other related batch group stored procedures:

- Reset Batch Group Jobs (CMXBG.RESET_BATCHGROUP)
- Get Batch Group Status Jobs (CMXBG.GET_BATCHGROUP_STATUS)

External Match Jobs

Matches “externally managed/prepared” records with an existing base object, yielding the results based on the current match settings—all without actually loading the data from the input table into the base object, changing data in the base object in any way, or changing the match table associated with the base object. You can use external matching to pretest data, test match rules, and inspect the results before running the actual Match job.

Note: The External Batch job executes as a batch job only—there is no corresponding SIF request that external applications can invoke.

Stored Procedure Definition for External Match Jobs

```
PROCEDURE CMXMA.EXTERNAL_MATCH(
  IN_ROWID_TABLE            IN CHAR(14)
, IN_USER_NAME              IN VARCHAR2(50)
, IN_MATCH_SET_NAME         IN VARCHAR2(500) DEFAULT NULL
, OUT_ERROR_MSG             OUT VARCHAR2(1024)
, RC                        OUT INT
, IN_JOB_GRP_CTRL           IN CHAR(14) DEFAULT NULL
, IN_JOB_GRP_ITEM           IN CHAR(14) DEFAULT NULL
)
```

Sample Job Execution Script for External Match Jobs

```
DECLARE
   IN_ROWID_TABLE           CHAR(14);
   IN_USER_NAME             VARCHAR2(50);
   IN_MATCH_SET_NAME        VARCHAR2(200);
   OUT_ERROR_MSG            VARCHAR2(1024);
   RC                       NUMBER;
BEGIN
   IN_ROWID_TABLE           := NULL;
   IN_USER_NAME             := NULL;
   IN_MATCH_SET_NAME        := NULL;
   OUT_ERROR_MSG            := NULL;
   RC                       := NULL;
   IN_JOB_GRP_CTRL          := NULL;
   IN_JOB_GRP_ITEM          := NULL;
   CMXMA.EXTERNAL_MATCH (IN_ROWID_TABLE, IN_USER_NAME, IN_MATCH_SET_NAME, OUT_ERROR_MSG, RC,
IN_JOB_GRP_CTRL, IN_JOB_GRP_ITEM);
   DBMS_OUTPUT.PUT_LINE ('OUT_ERROR_MESSAGE = ' || OUT_ERROR_MSG);
   DBMS_OUTPUT.PUT_LINE ('RC = ' || TO_CHAR(RC));
   IF RC < 0 THEN
      ROLLBACK;
   ELSE
      COMMIT;
   END IF;
END;
```

Generate Match Token Jobs

The Generate Match Tokens job runs the tokenize process, which generates match tokens and stores them in a match key table associated with the base object so that they can be used subsequently by the match process to identify candidates for matching.

You should run Generate Match Tokens jobs whenever match tokens need to be regenerated. Generate Match Tokens jobs apply to fuzzy-match base objects only—not to exact-match base objects.

Note: The Generate Match Tokens job generates the match tokens for the entire base object (when IN_FULL_RESTRIP_IND is set to 1). Check (select) the Re-generate All Match Tokens check box in the Batch Viewer to populate the IN_FULL_RESTRIP_IND parameter.

Identifiers for Executing Generate Match Token Jobs

Identifiers are used to execute the stored procedure associated with this batch job.

Dependencies for Generate Match Token Jobs

Each Generate Match Tokens job is dependent on the successful completion of the Load job responsible for loading data into the base object.

Successful Completion of Generate Match Token Jobs

Generate Match Tokens jobs must complete with a RUN_STATUS of 0 (Completed Successfully).

Stored Procedure Definition for Generate Match Token Jobs

```
PROCEDURE CMXMA.GENERATE_MATCH_TOKENS (
 IN_ROWID_TABLE             IN CHAR(14) --Rowid of a table
,IN_USER_NAME               IN VARCHAR2(50) --User name
,OUT_ERROR_MSG              OUT VARCHAR2(1024) --Error message, if any
,OUT_RETURN_CODE            OUT NUMBER --Return code (if no errors, 0 is returned)
,IN_JOB_GRP_CTRL            IN CHAR(14) DEFAULT NULL
,IN_JOB_GRP_ITEM            IN CHAR(14) DEFAULT NULL
```

```
,IN_FULL_RESTRIP_IND        IN NUMBER --Default 0, retokenize entire table if set to 1
(strip_truncate_insert)
)
```

Sample Job Execution Script for Generate Match Token Jobs

```
DECLARE
   IN_ROWID_TABLE           CHAR(14);
   IN_USER_NAME             VARCHAR2(50);
   OUT_ERROR_MSG            VARCHAR2(1024);
   OUT_RETURN_CODE          NUMBER;
   IN_FULL_RESTRIP_IND      NUMBER;
BEGIN
   IN_ROWID_TABLE           := NULL;
   IN_USER_NAME             := NULL;
   OUT_ERROR_MSG            := NULL;
   OUT_RETURN_CODE          := NULL;
   IN_FULL_RESTRIP_IND      := NULL;
   CMXMA.GENERATE_MATCH_TOKENS (IN_ROWID_TABLE, IN_USER_NAME, OUT_ERROR_MSG, OUT_RETURN_CODE,
IN_FULL_RESTRIP_IND);
   DBMS_OUTPUT.PUT_LINE ('OUT_ERROR_MESSAGE = ' || OUT_ERROR_MSG);
   DBMS_OUTPUT.PUT_LINE ('OUT_RETURN_CODE = ' || TO_CHAR(OUT_RETURN_CODE));
   IF OUT_RETURN_CODE < 0 THEN
      ROLLBACK;
   ELSE
      COMMIT;
   END IF;
END;
```

Get Batch Group Status Jobs

Get Batch Group Status jobs returns the status of a batch group.

Note that there are two other related batch group stored procedures:

- Execute Batch Group Jobs (CMXBG.EXECUTE_BATCHGROUP)
- Reset Batch Group Jobs (CMXBG.RESET_BATCHGROUP)

Hub Delete Jobs

The Hub Delete job removes specified data—up to and including an entire source system—from Informatica MDM Hub based on your XREF table input to the CMXDM.HUB_DELETE_BATCH stored procedure.

The XREF input table must contain either the primary key (ROWID_XREF) or the source system key (PKEY_SRC_OBJECT and ROWID_SYSTEM) of the cross-reference records to be deleted. If the XREF table contains both, the primary key and the source system key, then only the primary key is used to identify the records to be deleted. The Hub Delete job is faster when you use ROWID_XREF to identify the records to be deleted.

If you are deleting all records from a specific source system and cascade delete is enabled, then you must specify the XREFs of the specific source system as input to the cmxdm.hub_delete_batch stored procedure. All the parent records that belong to the specific source system are deleted but in the case of the child records that are cascaded to, XREFs and base objects that do not belong to the specified source system are also deleted.

After a Batch Delete job completes, temporary tables must be removed by using the following command:

```
cmxut.drop_table_in_list(OUT_TMP_TABLE_LIST,OUT_ERROR_MESSAGE,RC);
```

Note: The Hub Delete batch job will not delete the data if there are records queued for an Automerge job.

Do not run a Hub Delete job when there are automerge records in the match table. Run the Hub Delete job after the automerge matches are processed.

Cascade Delete

The Hub Delete job performs a *cascade delete* if you set the parameter IN_ALLOW_CASCADE_DELETE=1 for a base object in the stored procedure. With cascade delete, when records in the parent object are deleted, Hub Delete also removes the affected records in the child base object. Hub Delete checks each child base object table for related data that should be deleted given the removal of the parent base object record.

If a many-to-many parent child relationship exists between records, some parents may lose their child records during a cascde delete. Consider a scenario where two parent records share the same child record, and you delete one of the parent records. The associated child record is cascade deleted and the other parent record is left without its child record.

Note: For the prior example, the Hub Delete job may potentially delete XREF records from other source systems. To ensure that Hub Delete does not delete XREF records from other systems, do not use cascade delete. IN_ALLOW_CASCADE_DELETE forces Hub Delete to delete the child base objects and cross-references (regardless of system) when the parent base object is being deleted.

Note:

- If you do not set the IN_ALLOW_CASCADE_DELETE=1, Informatica MDM Hub generates an error message if there are child base objects referencing the deleted base objects record; Hub Delete fails, and Informatica MDM Hub performs a rollback operation for the associated data.
- IN_CASCADE_CHILD_SYSTEM_XREF=1 is not supported.

Hub Delete Impact on History Tables

The history tables that are impacted by the Hub Delete jobs are the ones, such as HIST, HXRF, that are created when history is enabled for base objects. Hub Delete jobs do not affect system-related merge history tables such as HMRG. Hub Delete jobs have the following impact on history tables:

- If you set IN_OVERRIDE_HISTORY_IND=1, Hub Delete writes to the history tables when deleting.
- If you set IN_OVERRIDE_HISTORY_IND=1 and set IN_PURGE_HISTORY_IND=1, then Hub Delete removes all traces of deleted records from the history tables.
- If you set IN_OVERRIDE_HISTORY_IND=0, then Hub Delete ignores IN_PURGE_HISTORY_IND and writes the last state of data into history tables when deleting (if History is ON).

Note: Informatica MDM Hub sets the HUB_STATE_IND to -9 in the HXRF when XREFs are deleted. The HIST table will be set to -9 if the base object record is deleted.

Hub Delete Impact on Records on Hold

The Hub Delete job removes “records on hold” or records that have had their CONSOLIDATION_IND column set to 9.

Stored Procedure Definition for Hub Delete Jobs

```
PROCEDURE CMXDM.HUB_DELETE_BATCH (
 IN_BO_TABLE_NAME              IN VARCHAR2(30)
,IN_XREF_LIST_TO_BE_DELETED    IN VARCHAR2(8)
,OUT_DELETED_XREF_COUNT        OUT INT
,OUT_DELETED_BO_COUNT          OUT INT
,OUT_ERROR_MSG                 OUT VARCHAR2(1024)
,OUT_RETURN_CODE               OUT INT
,OUT_TMP_TABLE_LIST            IN OUT VARCHAR2(32000)
,IN_RECALCULATE_BVT            IN INT DEFAULT 1
,IN_ALLOW_CASCADE_DELETE       IN INT DEFAULT 1
,IN_CASCADE_CHILD_SYSTEM_XREF  IN INT DEFAULT 0
,IN_OVERRIDE_HISTORY_IND       IN INT DEFAULT 0
,IN_PURGE_HISTORY_IND          IN INT DEFAULT 0
,IN_USER_NAME                  IN VARCHAR2(50) DEFAULT NULL
```

```
,IN_ALLOW_COMMIT_IND            IN INT DEFAULT 1
)
```

Parameters

Parameter	Description
IN_BO_TABLE_NAME	Name of the table that contains the list of base objects to delete.
IN_XREF_LIST_TO_BE_DELETED	Name of the table that contains the list of XREFs to delete.
IN_RECALCULATE_BVT_IND	If set to 1, recalculates BVT following base object and/or XREF delete.
IN_ALLOW_CASCADE_DELETE_IND	If set to 1, specifies that when records in the parent object are deleted, Hub Delete also removes the affected records in the child base object. Hub Delete checks each child base object table for related data that should be deleted given the removal of the parent base object record.
IN_CASCADE_CHILD_SYSTEM_XREF	This parameter is not supported. Leave the value for this parameter as the default, which is 0, when executing the procedure.
IN_OVERRIDE_HISTORY_IND	If set to 1, Hub Delete does not write to history tables when deleting. If you set IN_OVERRIDE_HISTORY_IND=1 and set IN_PURGE_HISTORY_IND=1, then Hub Delete removes the deleted records from the history tables.
IN_PURGE_HISTORY_IND	If set to 1, Hub Delete will remove all history records related to deleted XREF records that are not related to merge history. You cannot retrieve the deleted XREF record if required. If set to 0, Hub Delete will retain all history records related to deleted XREF records that are not related to merge history. You can retrieve the deleted XREF record if required. Default is 0.
IN_USER_NAME	Name of the HUB user who executes Batch Delete. The default value is `admin`.
IN_ALLOW_COMMIT_IND	If set to 1, Hub Delete commits or rolls back all changes. If set to 0, Hub Delete does not commit or roll back changes; user must explicitly commit or roll back changes.

Returns

Parameter	Description
OUT_DELETED_XREF_COUNT	Number of deleted XREFs.
OUT_DELETED_BO_COUNT	Number of deleted base objects.
OUT_TMP_TABLE_LIST	List of delimited tables that can be passed on to CMXUT.DROP_TEMP_TABLES stored procedure calls to clean up the temporary tables.
OUT_ERROR_MSG	Error message text.
OUT_RETURN_CODE	Error code. If zero (0), then the stored procedure completed successfully. The procedure will return a non-zero value in case of an error.

Sample Job Execution Script for Hub Delete Jobs

```
DECLARE
  IN_BO_TABLE_NAME             VARCHAR2(40);
  IN_XREF_LIST_TO_BE_DELETED   VARCHAR2(40);
  IN_RECALCULATE_BVT_IND       NUMBER;
  IN_ALLOW_CASCADE_DELETE      NUMBER;
  IN_CASCADE_CHILD_SYSTEM_XREF NUMBER;
  IN_OVERRIDE_HISTORY_IND      NUMBER;
  IN_PURGE_HISTORY_IND         NUMBER;
  IN_USER_NAME                 VARCHAR2(100);
  IN_ALLOW_COMMIT_IND          NUMBER;
  OUT_DELETED_XREF_COUNT       NUMBER;
  OUT_DELETED_BO_COUNT         NUMBER;
  OUT_TMP_TABLE_LIST           VARCHAR2(32000);
  OUT_ERROR_MESSAGE            VARCHAR2(1024);
  OUT_RETURN_CODE              NUMBER;
BEGIN
  IN_BO_TABLE_NAME             := 'C_CUSTOMER';
  IN_XREF_LIST_TO_BE_DELETED   := 'TMP_DELETE_KEYS';
  OUT_DELETED_XREF_COUNT       := NULL;
  OUT_DELETED_BO_COUNT         := NULL;
  OUT_TMP_TABLE_LIST           := NULL;
  OUT_ERROR_MESSAGE            := NULL;
  OUT_RETURN_CODE              := NULL;
  IN_RECALCULATE_BVT_IND       := 1;
  IN_ALLOW_CASCADE_DELETE      := 1;
  IN_CASCADE_CHILD_SYSTEM_XREF := 0;
  IN_OVERRIDE_HISTORY_IND      := 0;
  IN_PURGE_HISTORY_IND         := 0;
  IN_USER_NAME                 := 'ADMIN';
  IN_ALLOW_COMMIT_IND          := 0;

  --
  BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE ' || IN_XREF_LIST_TO_BE_DELETED;
    EXCEPTION WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE(' SQLERRM = ' || SQLERRM);
  END;
  --
  EXECUTE IMMEDIATE 'CREATE TABLE ' || IN_XREF_LIST_TO_BE_DELETED
                || ' AS SELECT PKEY_SRC_OBJECT, ROWID_SYSTEM FROM '
                || IN_BO_TABLE_NAME
                || '_XREF WHERE ROWID_SYSTEM = ''SALES''';
  --
  CMXDM.HUB_DELETE_BATCH(IN_BO_TABLE_NAME,
                         IN_XREF_LIST_TO_BE_DELETED,
                         OUT_DELETED_XREF_COUNT,
                         OUT_DELETED_BO_COUNT,
                         OUT_ERROR_MESSAGE,
                         OUT_RETURN_CODE,
                         OUT_TMP_TABLE_LIST,
                         IN_RECALCULATE_BVT_IND,
                         IN_ALLOW_CASCADE_DELETE,
                         IN_CASCADE_CHILD_SYSTEM_XREF,
                         IN_OVERRIDE_HISTORY_IND,
                         IN_PURGE_HISTORY_IND,
                         IN_USER_NAME,
                         IN_ALLOW_COMMIT_IND);
  DBMS_OUTPUT.PUT_LINE(' RETURN CODE IS ' || OUT_RETURN_CODE);
  DBMS_OUTPUT.PUT_LINE(' MESSAGE IS ' || OUT_ERROR_MESSAGE);
  DBMS_OUTPUT.PUT_LINE(' XREF RECORDS DELETED: ' || OUT_DELETED_XREF_COUNT);
  DBMS_OUTPUT.PUT_LINE(' BO RECORDS DELETED: ' || OUT_DELETED_BO_COUNT);

  IF OUT_RETURN_CODE < 0 THEN
    ROLLBACK;
  ELSE
    COMMIT;
  END IF;

  CMXUT.DROP_TABLE_IN_LIST(OUT_TMP_TABLE_LIST,
```

```
                              OUT_ERROR_MESSAGE,
                              OUT_RETURN_CODE); --clean up temporary tables
END;
/
```

Key Match Jobs

Key Match jobs are used to match records from two or more sources when these sources use the same primary key.

Key Match jobs compare new records to each other and to existing records, and identifies potential matches based on the comparison of source record keys as defined by the match rules.

Identifiers for Executing Key Match Jobs

Identifiers are used to execute the stored procedure associated with this batch job.

Dependencies for Key Match Jobs

Key Match jobs are dependent on the successful completion of the Load job responsible for loading data into the base object.

The Key Match job cannot have been run after any changes were made to the data.

Successful Completion of Key Match Jobs

Key Match jobs must complete with a RUN_STATUS of 0 (Completed Successfully).

Stored Procedure Definition for Key Match Jobs

```
PROCEDURE CMXMA.KEY_MATCH (
 IN_ROWID_TABLE             IN CMXLB.CMX_ROWID
,IN_USER_NAME               IN CMXLB.CMX_USER_NAME
,OUT_ERROR_MSG              OUT CMXLB.CMX_MESSAGE
,RC                         OUT INT
,IN_JOB_GRP_CTRL            IN CMXLB.CMX_ROWID DEFAULT NULL
,IN_JOB_GRP_ITEM            IN CMXLB.CMX_ROWID DEFAULT NULL
);
```

Sample Job Execution Script for Key Match Jobs

```
DECLARE
   IN_ROWID_TABLE           VARCHAR2(14);
   IN_USER_NAME             VARCHAR2(50);
   OUT_ERROR_MESSAGE        VARCHAR2(1024);
   OUT_RETURN_CODE          NUMBER;
BEGIN
   IN_ROWID_TABLE           := NULL;
   IN_USER_NAME             := 'myusername';
   OUT_ERROR_MESSAGE        := NULL;
   OUT_RETURN_CODE          := NULL;
   CMXMA.KEY_MATCH (IN_ROWID_TABLE, IN_USER_NAME, OUT_ERROR_MESSAGE, OUT_RETURN_CODE);
   DBMS_OUTPUT.PUT_LINE (' Row id table = ' || IN_ROWID_TABLE);
   CMXMA.KEY_MATCH ( IN_ROWID_TABLE, IN_USER_NAME, OUT_ERROR_MESSAGE, OUT_RETURN_CODE);
   DBMS_OUTPUT.PUT_LINE ('OUT_ERROR_MESSAGE = ' || OUT_ERROR_MESSAGE);
   DBMS_OUTPUT.PUT_LINE ('OUT_RETURN_CODE = ' || TO_CHAR(OUT_RETURN_CODE));
   IF OUT_RETURN_CODE < 0 THEN
      ROLLBACK;
   ELSE
      COMMIT;
   END IF;
END;
```

Load Jobs

Load jobs move data from staging tables to the final target objects, and apply any trust and validation rules where appropriate.

Identifiers for Executing Load Jobs

Identifiers are used to execute the stored procedure associated with this batch job.

Dependencies for Load Jobs

Each Load job is dependent on the success of the Stage job that precedes it.

In addition, each Load job is governed by the demands of referential integrity constraints and is dependent on the successful completion of all other Load jobs responsible for populating tables referenced by the base object that is the target of the load. Run the loads for parent tables before the loads for child tables.

Successful Completion of Load Jobs

A Load job must complete with a RUN_STATUS of 0 (Completed Successfully) or 1 (Completed with Errors) to be considered successful. The Auto Match and Merge jobs for a target base object can either be run on successful completion of each Load job, or on successful completion of all Load jobs for the base object.

Stored Procedure Definition for Load Jobs

```
PROCEDURE LOAD_MASTER (
 IN_STG_ROWID_TABLE         IN CHAR(14) --Rowid of staging table
,IN_USER_NAME               IN VARCHAR2(50) --Database user name
,OUT_ERROR_MSG              OUT VARCHAR2 (1024) --Error message, if any
,OUT_RETURN_CODE            OUT NUMBER --Return code (if no errors, 0 is returned)
,IN_FORCE_UPDATE_IND        --Forced update value Default 0, 1 for Forced update
,IN_ROWID_JOB_GRP_CTRL      IN CHAR(14)
,IN_ROWID_JOB_GRP_ITEM      IN CHAR(14)
)
```

Sample Job Execution Script for Load Jobs

```
DECLARE
   IN_STG_ROWID_TABLE       CHAR(14);
   IN_USER_NAME             VARCHAR2(50);
   OUT_ERROR_MSG            VARCHAR2(1024);
   OUT_RETURN_CODE          NUMBER;
   IN_FORCE_UPDATE_IND      NUMBER;
   IN_ROWID_JOB_GRP_CTRL    CHAR(14);
   IN_ROWID_JOB_GRP_ITEM    CHAR(14);
BEGIN
   IN_STG_ROWID_TABLE       := 'SVR1.1L9';
   IN_USER_NAME             := 'ADMIN';
   IN_ROWID_JOB_GRP_CTRL    := NULL;
   IN_ROWID_JOB_GRP_ITEM    := NULL;
   OUT_ERROR_MSG            := NULL;
   OUT_RETURN_CODE          := NULL;
   IN_FORCE_UPDATE_IND      := 1;
   CMXLD.LOAD_MASTER (IN_STG_ROWID_TABLE, IN_USER_NAME, OUT_ERROR_MSG,
OUT_RETURN_CODE,IN_FORCE_UPDATE_IND, IN_ROWID_JOB_GRP_CTRL, IN_ROWID_JOB_GRP_ITEM);
   DBMS_OUTPUT.PUT_LINE ('OUT_ERROR_MSG = ' || OUT_ERROR_MSG);
   DBMS_OUTPUT.PUT_LINE ('OUT_RETURN_CODE = ' || TO_CHAR(OUT_RETURN_CODE));
   IF OUT_RETURN_CODE < 0 THEN
     ROLLBACK;
   ELSE
     COMMIT;
   END IF;
END;
```

Manual Link Jobs

Manual Link jobs execute manually linking in the Merge Manager tool. Manual Link jobs are used with link-style base objects only. Results are stored in the _LINK table.

Manual Merge Jobs

After the Match job has been run, data stewards can use the Merge Manager to process records that have been queued by a Match job for manual merge.

Manual Merge jobs are run in the Merge Manager—not in the Batch Viewer. The Batch Viewer only allows you to inspect job execution logs for Manual Merge jobs that were run in the Merge Manager.

Stored Procedure Definition for Manual Merge Jobs

```
PROCEDURE CMXMM.MERGE(
 IN_ROWID_TABLE             CMXLB.CMX_ROWID
,IN_SRC_ROWID_OBJECT        CMXLB.CMX_ROWID
,IN_TGT_ROWID_OBJECT        CMXLB.CMX_ROWID
,IN_ROWID_MATCH_RULE        CMXLB.CMX_ROWID
,IN_AUTOMERGE_IND           INT
,IN_PROMOTE_STRING          CMXLB.CMX_MED_STR --
Sample:SVR1.3EL~1~SVR1.3EJ~2~SVR1.3EX~3~SVR1.3EM~1000903010200~
,IN_ROWID_JOB               CMXLB.CMX_ROWID
,IN_INTERACTION_ID          INT
,IN_USER_NAME               CMXLB.CMX_USER_NAME
,OUT_MERGED_IS_UNIQUE_IND   OUT INT
,OUT_ERROR_MESSAGE          OUT CMXLB.CMX_MESSAGE
,OUT_RETURN_CODE            OUT INT
,CALLED_MANUALLY_IND        IN INT DEFAULT 1
,OUT_TMP_TABLE_LIST         OUT CMXLB.CMX_BIG_STR
,IN_ROWID_TASK              IN CMXLB.CMX_ROWID DEFAULT NULL
);
```

Sample Job Execution Script for Manual Merge Jobs

```
DECLARE
    V_ROWID_TABLE              CHAR(14);
    V_SRC_ROWID_OBJECT         CHAR(14);
    V_TGT_ROWID_OBJECT         CHAR(14);
    V_PROMOTE_STRING           VARCHAR2(2000);
    V_INTERACTION_ID           INT      := NULL;
    V_USER_NAME                VARCHAR2(50);
    OUT_MERGED_IS_UNIQUE_IND   INT;
    OUT_ERROR_MESSAGE          VARCHAR2(2000);
    OUT_RETURN_CODE            INT;
    OUT_TMP_TABLE_LIST         VARCHAR2(32000);

BEGIN
    SELECT ROWID_TABLE
    INTO   V_ROWID_TABLE
    FROM   C_REPOS_TABLE
    WHERE  TABLE_NAME = 'C_CUSTOMER';
    V_TGT_ROWID_OBJECT         := 1;
    V_SRC_ROWID_OBJECT         := 2;
    V_PROMOTE_STRING           := NULL;   --Contains Rowid_column~winner~ For trusted columns
to force the winning cell for that column.
                                --Winner can either be "s"ource or "t"arget.  Example:
'svr1.7sv~t~svr1.7sw~s~'
    V_INTERACTION_ID           := NULL;
    OUT_TMP_TABLE_LIST         :=NULL;
    CMXMM.MANUAL_MERGE (V_ROWID_TABLE, V_SRC_ROWID_OBJECT, V_TGT_ROWID_OBJECT,
V_PROMOTE_STRING, V_INTERACTION_ID, 'ADMIN', OUT_MERGED_IS_UNIQUE_IND, OUT_ERROR_MESSAGE,
OUT_RETURN_CODE, OUT_TMP_TABLE_LIST);
    DBMS_OUTPUT.PUT_LINE( 'MERGED IS UNIQUE IND: ' || OUT_MERGED_IS_UNIQUE_IND );
    DBMS_OUTPUT.PUT_LINE( 'RETURN MESSAGE: ' || SUBSTR( OUT_ERROR_MESSAGE, 1, 255 ));
```

```
    DBMS_OUTPUT.PUT_LINE( 'RETURN CODE: ' || OUT_RETURN_CODE );
    DBMS_OUTPUT.PUT_LINE('OUT_ERROR_MESSAGE = ' || OUT_ERROR_MESSAGE);
    DBMS_OUTPUT.PUT_LINE('OUT_RETURN_CODE = ' || TO_CHAR(OUT_RETURN_CODE));
    IF OUT_RETURN_CODE < 0 THEN
      ROLLBACK;
    ELSE
      COMMIT;
    END IF;
    CMXUT.DROP_TABLE_IN_LIST(OUT_TMP_TABLE_LIST,OUT_ERROR_MESSAGE,OUT_RETURN_CODE); --Clean up
temporary tables
END;
```

Manual Unlink Jobs

Manual Unlink jobs execute manually unlinking of records that were previously linked manually in the Merge Manager tool or through one of these stored procedure jobs.

Manual Unmerge Jobs

The Unmerge job can unmerge already-consolidated records, whether those records were consolidated using Automerge, Manual Merge, manual edit, Load by Rowid_Object, or Put Xref. The Unmerge job succeeds or fails as a single transaction: if the server fails while the Unmerge job is executing, the unmerge process is rolled back.

Cascade Unmerge

The Unmerge job performs a *cascade unmerge* if this feature is enabled for this base object in the Schema Manager in the Hub Console. With cascade unmerge, when records in the parent object are unmerged, Informatica MDM Hub also unmerges affected records in the child base object.

This feature applies to unmerging records across base objects. This is configured per base object (using the **Unmerge Child When Parent Unmerges** check box on the Merge Settings tab in the Schema Manager). Cascade unmerge applies only when a foreign-key relationship exists between two base objects.

For example: Customer A record (parent) in the Customer base object has multiple address records (children) in the Address base object. The two tables are linked by a unique key (Customer_ID).

- **When cascade unmerge is enabled**—Unmerging the parent record (Customer A) in the Customer base object also unmerges Customer A's child address records in the Address base object.
- **When cascade unmerge is disabled**—Unmerging the parent record (Customer A) in the Customer base object has no effect on Customer A's child records in the Address base object; they are NOT unmerged.

Unmerging All Records or One Record

In your job execution script, you can specify the scope of records to unmerge by setting IN_UNMERGE_ALL_XREFS_IND.

- IN_UNMERGE_ALL_XREFS_IND=0: Default setting. Unmerges the single record identified in the specified XREF to its state prior to the merge.
- IN_UNMERGE_ALL_XREFS_IND=1: Unmerges all XREFs to their state prior to the merge. Use this option to quickly unmerge all XREFs for a single consolidated record in a single operation.

Linear and Tree Unmerge

These features apply to unmerging contributing records from within a single base object. There is a hierarchy of merges consisting of a root (top of the tree, or BVT), branches (merged records), and leaves (the original contributing records at end of the branches). This hierarchy can be many levels deep.

In your job execution script, you can specify the type of unmerge (linear or tree unmerge) by setting IN_TREE_UNMERGE_IND:

- IN_TREE_UNMERGE_IND=0: Default setting. Linear Unmerge
- IN_TREE_UNMERGE_IND=1: Tree Unmerge

Linear Unmerge

Linear unmerge is the default behavior. During a linear unmerge, a base object record is unmerged and taken out of the existing merge tree structure. Only the unmerged base object record itself will come out the merge tree structure, and all base object records below it in the merge tree will stay in the original merge tree.

Tree Unmerge

Tree unmerge is an alternative to linear unmerge. A tree of merged base object records is a hierarchical structure of the merge history, reflecting the sequence of merge operations that have occurred. Merge history is kept during the merge process in these tables:

- The cross-reference table tracks the ROWID_OBJECT of the record when it was first loaded into the Hub in the ORIG_ROWID_OBJECT column. The ORIG_ROWID_OBJECT is the ROWID_OBJECT that the record reverts to after an unmerge.
- The HMRG table provides a hierarchical view of the merge history, a tree of merged base object records, in addition to an interactive unmerge history.

During a tree unmerge, you unmerge a tree of merged base object records as an intact sub-structure. A sub-tree having unmerged base object records as root will come out from the original merge tree structure. (For example, merge a1 and a2 into a, then merge b1 and b2 into b, and then finally merge a and b into c. If you then perform a tree unmerge on a, and then unmerge a from a1, a2 is a sub tree and will come out from the original tree c. As a result, a is the root of the tree after the unmerge.)

Identifiers for Executing Manual Unmerge Jobs

Identifiers are used to execute the stored procedure associated with this batch job.

Committing Unmerge Transactions After Error Checking

The Unmerge stored procedure is transaction-enabled and can be rolled back if an error occurs during execution.

Note: After calling the Unmerge, check the return code (OUT_RETURN_CODE) in your error handling.

- If any failure occurred during execution (OUT_RETURN_CODE <> 0), *immediately* roll back any changes. Wait until *after* you have successfully rolled back the unmerge changes before you invoke Unmerge again.
- If no failure occurred during execution (OUT_RETURN_CODE = 0), *commit* any unmerge changes.

Dependencies for Manual Unmerge Jobs

Each Manual Unmerge job is dependent on data having already been merged.

Successful Completion of Manual Unmerge Jobs

A Manual Unmerge job must complete with a RUN_STATUS of 0 (Completed Successfully) or 1 (Completed with Errors) to be considered successful.

Note: The CMXUT.DROP_TEMP_TABLES stored procedure must be executed after any procedure that returns OUT_TMP_TABLE_LIST in order to clean the temporary tables.

Stored Procedure Definition for Manual Unmerge Jobs

```
PROCEDURE CMXMM.UNMERGE (
 IN_ROWID_TABLE             CMXLB.CMX_ROWID
,IN_ROWID_SYSTEM            CMXLB.CMX_ROWID
,IN_PKEY_SRC_OBJECT         CMXLB.CMX_PKEY_SRC_OBJECT
,IN_TREE_UNMERGE_IND        INT DEFAULT 0
,IN_ROWID_JOB_CTL           CMXLB.CMX_ROWID
,IN_INTERACTION_ID          INT
,IN_USER_NAME               CMXLB.CMX_USER_NAME
,OUT_UNMERGED_ROWID         OUT CMXLB.CMX_ROWID
,OUT_TMP_TABLE_LIST         OUT CLOB
,OUT_ERROR_MESSAGE          OUT CMXLB.CMX_MESSAGE
,OUT_RETURN_CODE            OUT INT
,IN_UNMERGE_ALL_XREFS_IND   IN INT DEFAULT 0)
```

Sample Job Execution Script for Manual Unmerge Jobs

```
DECLARE
   IN_ROWID_TABLE            CHAR (14);
   IN_ROWID_SYSTEM           CHAR (14);
   IN_PKEY_SRC_OBJECT        VARCHAR2 (255);
   IN_TREE_UNMERGE_IND       NUMBER;
   IN_ROWID_JOB_CTL          CHAR (14);
   IN_INTERACTION_ID         NUMBER;
   IN_USER_NAME              VARCHAR2 (50);
   OUT_UNMERGED_ROWID        CHAR (14);
   OUT_TMP_TABLE_LIST        VARCHAR2 (32000);
   OUT_ERROR_MESSAGE         VARCHAR2 (1024);
   RC                        NUMBER;
   IN_UNMERGE_ALL_XREFS_IND  NUMBER;
BEGIN
   IN_ROWID_TABLE            := 'SVR1.8ZC';
   IN_ROWID_SYSTEM           := 'SVR1.7NJ';
   IN_PKEY_SRC_OBJECT        := '6';
   IN_TREE_UNMERGE_IND       := 0;      -- Default 0, 1 for tree unmerge
   IN_ROWID_JOB_CTL          := NULL;
   IN_INTERACTION_ID         := NULL;
   IN_USER_NAME              := 'XHE';
   OUT_UNMERGED_ROWID        := NULL;
   OUT_TMP_TABLE_LIST        := NULL;
   OUT_ERROR_MESSAGE         := NULL;
   RC                        := NULL;
   IN_UNMERGE_ALL_XREFS_IND := 0; -- default 0, 1 for unmerge_all
   CMXMM.UNMERGE ( IN_ROWID_TABLE, IN_ROWID_SYSTEM, IN_PKEY_SRC_OBJECT, IN_TREE_UNMERGE_IND,
IN_ROWID_JOB_CTL, IN_INTERACTION_ID, IN_USER_NAME, OUT_UNMERGED_ROWID, OUT_TMP_TABLE_LIST,
OUT_ERROR_MESSAGE, RC, IN_UNMERGE_ALL_XREFS_IND );
   DBMS_OUTPUT.PUT_LINE (' Return Code = ' || rc);
   DBMS_OUTPUT.PUT_LINE (' Message is = ' || out_error_message);
   DBMS_OUTPUT.PUT_LINE ('OUT_ERROR_MESSAGE = ' || OUT_ERROR_MESSAGE);
   DBMS_OUTPUT.PUT_LINE ('RC = ' || TO_CHAR(RC));
   IF RC < 0 THEN
      ROLLBACK;
   ELSE
      COMMIT;
   END IF;
   CMXUT.DROP_TABLE_IN_LIST(OUT_TMP_TABLE_LIST,OUT_ERROR_MESSAGE,RC); --Clean up temporary
tables
END;
```

Match Jobs

Match jobs find duplicate records in the base object, based on the current match rules.

Note: Do not run a Match job on a base object that is used to define relationships between records in inter-table or intra-table match paths. Doing so will change the relationship data, resulting in the loss of the associations between records.

Identifiers for Executing Match Jobs

Identifiers are used to execute the stored procedure associated with this batch job.

Dependencies for Match Jobs

Each Match job is dependent on new / updated records in the base object that have been tokenized and are thus queued for matching. For parent base objects that have children, the Match job is also dependent on the successful completion of the data tokenization jobs for all child tables, which in turn is dependent on successful Load jobs for the child tables.

Successful Completion of Match Jobs

Match jobs must complete with a RUN_STATUS of 0 (Completed Successfully) or 1 (Completed with Errors) to be considered successful.

Stored Procedure for Match Jobs

```
PROCEDURE CMXMA.MATCH (
 IN_ROWID_TABLE            IN CMXLB.CMX_ROWID
,IN_USER_NAME              IN CMXLB.CMX_USER_NAME
,OUT_ERROR_MSG             OUT CMXLB.CMX_MESSAGE
,RC                        OUT INT
,IN_VALIDATE_TABLE_NAME    IN CMXLB.CMX_TABLE_NAME DEFAULT NULL --Table of validation rowid's
(can include rowid_match_rule)
,IN_MATCH_ANALYZE_IND      IN INT DEFAULT 0 --Match analyze to check for match data
,IN_MATCH_SET_NAME         IN CMXLB.CMX_SMALL_STR DEFAULT NULL
,IN_JOB_GRP_CTRL           IN CMXLB.CMX_ROWID DEFAULT NULL
,IN_JOB_GRP_ITEM           IN CMXLB.CMX_ROWID DEFAULT NULL
);
```

Sample Job Execution Script for Match Jobs

```
DECLARE
   IN_ROWID_TABLE           CHAR(14);
   IN_USER_NAME             VARCHAR2(50);
   OUT_ERROR_MSG            VARCHAR2(1024);
   RC                       NUMBER;
   IN_VALIDATE_TABLE_NAME   VARCHAR2(30);
   IN_MATCH_ANALYZE_IND     NUMBER;
   IN_MATCH_SET_NAME        VARCHAR2(500);
   IN_JOB_GRP_CTRL          CHAR(14);
   IN_JOB_GRP_ITEM          CHAR(14);
BEGIN
   IN_ROWID_TABLE          := NULL;
   IN_USER_NAME            := NULL;
   OUT_ERROR_MSG           := NULL;
   RC                      := NULL;
   IN_VALIDATE_TABLE_NAME  := NULL;
   IN_MATCH_ANALYZE_IND    := NULL;
   IN_MATCH_SET_NAME       := NULL;
   IN_JOB_GRP_CTRL         := NULL;
   IN_JOB_GRP_ITEM         := NULL;
   CMXMA.MATCH ( IN_ROWID_TABLE, IN_USER_NAME, OUT_ERROR_MSG, RC, IN_VALIDATE_TABLE_NAME,
IN_MATCH_ANALYZE_IND, IN_MATCH_SET_NAME, IN_JOB_GRP_CTRL, IN_JOB_GRP_ITEM );
   DBMS_OUTPUT.PUT_LINE ('OUT_ERROR_MESSAGE = ' || OUT_ERROR_MESSAGE);
   DBMS_OUTPUT.PUT_LINE ('RC = ' || TO_CHAR(RC));
   IF RC < 0 THEN
      ROLLBACK;
   ELSE
      COMMIT;
   END IF;
END;
```

Match Analyze Jobs

Match Analyze jobs perform a search to gather metrics about matching without conducting any actual matching.

Match Analyze jobs are typically used to fine-tune match rules.

Identifiers for Executing Match Analyze Jobs

Identifiers are used to execute the stored procedure associated with this batch job.

Dependencies for Match Analyze Jobs

Each Match Analyze job is dependent on new / updated records in the base object that have been tokenized and are thus queued for matching. For parent base objects, the Match Analyze job is also dependent on the successful completion of the data tokenization jobs for all child tables, which in turn is dependent on successful Load jobs for the child tables.

Successful Completion of Match Analyze Jobs

Match Analyze jobs must complete with a RUN_STATUS of 0 (Completed Successfully) or 1 (Completed with Errors) to be considered successful.

Stored Procedure for Match Analyze Jobs

```
PROCEDURE CMXMA.MATCH (
IN_ROWID_TABLE IN CMXLB.CMX_ROWID
,IN_USER_NAME IN CMXLB.CMX_USER_NAME
,OUT_ERROR_MSG OUT CMXLB.CMX_MESSAGE
,RC OUT INT
,IN_VALIDATE_TABLE_NAME IN CMXLB.CMX_TABLE_NAME DEFAULT NULL --Table of validation rowid's (can
include rowid_match_rule)
,IN_MATCH_ANALYZE_IND IN INT DEFAULT 0 --Match analyze to check for match data
,IN_MATCH_SET_NAME IN CMXLB.CMX_SMALL_STR DEFAULT NULL
,IN_JOB_GRP_CTRL IN CMXLB.CMX_ROWID DEFAULT NULL
,IN_JOB_GRP_ITEM IN CMXLB.CMX_ROWID DEFAULT NULL
);
```

Sample Job Execution Script for Match Analyze Jobs

```
DECLARE
   IN_ROWID_TABLE          CHAR(14);
   IN_USER_NAME            VARCHAR2(50);
   OUT_ERROR_MSG           VARCHAR2(1024);
   OUT_RETURN_CODE         NUMBER;
   IN_VALIDATE_TABLE_NAME  VARCHAR2(30);
   IN_MATCH_ANALYZE_IND    NUMBER;
BEGIN
   IN_ROWID_TABLE          := NULL;
   IN_USER_NAME            := NULL;
   OUT_ERROR_MSG           := NULL;
   OUT_RETURN_CODE         := NULL;
   IN_VALIDATE_TABLE_NAME  := NULL;
   IN_MATCH_ANALYZE_IND    := 1;
   CMXMA.MATCH (IN_ROWID_TABLE, IN_USER_NAME, OUT_ERROR_MSG, OUT_RETURN_CODE,
IN_VALIDATE_TABLE_NAME, IN_MATCH_ANALYZE_IND );
   DBMS_OUTPUT.PUT_LINE ('OUT_ERROR_MESSAGE = ' || OUT_ERROR_MESSAGE);
   DBMS_OUTPUT.PUT_LINE ('OUT_RETURN_CODE = ' || TO_CHAR(OUT_RETURN_CODE));
   IF OUT_RETURN_CODE < 0 THEN
      ROLLBACK;
   ELSE
      COMMIT;
   END IF;
END;
```

Match for Duplicate Data Jobs

A Match for Duplicate Data job searches for exact duplicates to consider them matched.

Use it to manually run the Match for Duplicate Data process when you want to use your own rule as the match for duplicates criteria instead of all the columns in the base object. The maximum number of exact duplicates is based on the base object columns defined in the Duplicate Match Threshold property in the Schema Manager for each base object.

Note: The Match for Duplicate Data batch job has been deprecated.

Identifiers for Executing Match for Duplicate Jobs

Identifiers are used to execute the stored procedure associated with this batch job.

Dependencies for Match for Duplicate Data Jobs

Match for Duplicate Data jobs require the existence of unconsolidated data in the base object.

Successful Completion of Match for Duplicate Data Jobs

Match for Duplicate Data jobs must complete with a RUN_STATUS of 0 (Completed Successfully).

Stored Procedure Definition for Match for Duplicate Data Jobs

```
PROCEDURE CMXMA.MATCH_FOR_DUPS (
 IN_ROWID_TABLE             IN CMXLB.CMX_ROWID --Rowid of a table
,IN_USER_NAME               IN CMXLB.CMX_USER_NAME --User name
,OUT_ERROR_MSG              OUT CMXLB.CMX_MESSAGE --Error message, if any
,OUT_RETURN_CODE            OUT INT --Return code (if no errors, 0 is returned)
,IN_JOB_GRP_CTRL            IN CMXLB.CMX_ROWID DEFAULT NULL
,IN_JOB_GRP_ITEM            IN CMXLB.CMX_ROWID DEFAULT NULL
);
```

Sample Job Execution Script for Match for Duplicate Data Jobs

```
DECLARE
   IN_ROWID_TABLE           CHAR(14);
   IN_USER_NAME             VARCHAR2(200);
   OUT_ERROR_MSG            VARCHAR2(2000);
   OUT_RETURN_CODE          NUMBER;
BEGIN
   IN_ROWID_TABLE           := NULL;
   IN_USER_NAME             := NULL;
   OUT_ERROR_MSG            := NULL;
   OUT_RETURN_CODE          := NULL;
   CMXMA.MATCH_FOR_DUPS ( IN_ROWID_TABLE, IN_USER_NAME, OUT_ERROR_MSG, OUT_RETURN_CODE);
   DBMS_OUTPUT.PUT_LINE ('OUT_ERROR_MESSAGE = ' || OUT_ERROR_MESSAGE);
   DBMS_OUTPUT.PUT_LINE ('OUT_RETURN_CODE = ' || TO_CHAR(OUT_RETURN_CODE));
   IF OUT_RETURN_CODE < 0 THEN
      ROLLBACK;
   ELSE
      COMMIT;
   END IF;
END;
```

Multi Merge Jobs

Multi Merge jobs allow the merge of multiple records in a single job. This batch job is initiated only by external applications that invoke the SIF MultiMergeRequest.

The Multi Merge stored procedure performs the following functions:

- Calls group_merge based on the incoming list of base object records
- Uses PUT_XREF to process user-selected winning values into the base object record

When executing the Multi Merge stored procedure:

- Merge rowid_objects in the IN_MEMBER_ROWID_LIST into IN_SURVIVING_ROWID have the column values provided from IN_VAL_LIST as the base object's winning cell values. Values are delimited by ~. For example: val1~val2~val3~
- The first rowid_object in IN_MEMBER_ROWID_LIST will be selected as the surviving rowid_object if the IN_SURVIVING_ROWID is not provided.
- If IN_MEMBER_ROWID_LIST is NULL, IN_SURVIVING_ROWID will be considered as group_id in the link table. In this case, all active member rowid_objects belonging to this group_id will be merged into IN_SURVIVING_ROWID.
- Values in the IN_MEMBER_ROWID_LIST, IN_COL_LIST, and IN_VAL_LIST columns are delimited by '~'. For example: value1~value2~value3~

Identifiers for Executing Multi Merge Jobs

Identifiers are used to execute the stored procedure associated with this batch job.

Dependencies for Multi Merge Jobs

Each Multi Merge job is dependent on the successful completion of the match process for this base object.

Successful Completion of Multi Merge Jobs

Multi Merge jobs must complete with a RUN_STATUS of 0 (Completed Successfully) or 1 (Completed with Errors) to be considered successful.

Stored Procedure Definition for Multi Merge Jobs

```
PROCEDURE CMXMM.MULTI_MERGE (
IN_ROWID_TABLE CMXLB.CMX_ROWID
,IN_SURVIVING_ROWID CMXLB.CMX_ROWID
,IN_MEMBER_ROWID_LIST CMXLB.CMX_BIG_STR --delimited by '~'
,IN_ROWID_MATCH_RULE CMXLB.CMX_ROWID
,IN_COL_LIST CMXLB.CMX_BIG_STR --delimited by '~'
,IN_VAL_LIST CMXLB.CMX_BIG_STR --delimited by '~'
,IN_INTERACTION_ID INT
,IN_USER_NAME CMXLB.CMX_USER_NAME
,OUT_TMP_TABLE_LIST IN OUT CMXLB.CMX_BIG_STR
,OUT_ERROR_MESSAGE OUT CMXLB.CMX_MESSAGE
,OUT_RETURN_CODE OUT INT
,IN_WINNING_CELL_OVERRIDE IN CMXLB.CMX_MED_STR DEFAULT NULL
,IN_ROWID_TASK IN CMXLB.CMX_ROWID DEFAULT NULL
);
```

Sample Job Execution Script for Multi Merge Jobs

```
DECLARE
   IN_ROWID_TABLE            CHAR(14);
   IN_SURVIVING_ROWID        CHAR(14);
   IN_MEMBER_ROWID_LIST      VARCHAR2(4000);
   IN_ROWID_MATCH_RULE       VARCHAR2(4000);
   IN_COL_LIST               VARCHAR2(4000);
   IN_VAL_LIST               VARCHAR2(4000);
   IN_INTERACTION_ID         NUMBER;
```

```
   IN_USER_NAME              VARCHAR2(200);
   IN_WINNING_CELL_OVERRIDE  VARCHAR2(4000);
   OUT_ERROR_MESSAGE         VARCHAR2(200);
   OUT_RETURN_CODE           NUMBER;
   OUT_TMP_TABLE_LIST        VARCHAR2(32000);
BEGIN
   IN_ROWID_TABLE            := 'SVR1.CP4      ';
   IN_SURVIVING_ROWID        := '40            ';
   IN_MEMBER_ROWID_LIST      := '42           ~44            ~45            ~47
~48          ~49            ~';
   IN_ROWID_MATCH_RULE       := NULL;
   IN_COL_LIST               := 'SVR1.CSB      ~SVR1.CSE      ~SVR1.CSG      ~SVR1.CSH
~SVR1.CSA      ~';
   IN_VAL_LIST               := 'INDU~THOMAS~11111111111~F~1000~';
   IN_INTERACTION_ID         := 0;
   IN_USER_NAME              := 'INDU';
   IN_WINNING_CELL_OVERRIDE  := NULL
   OUT_ERROR_MESSAGE         := NULL;
   OUT_RETURN_CODE           := NULL;
   CMXMM.MULTI_MERGE (IN_ROW_TABLE, IN_SURVIVING_ROWID, IN_MEMBER_ROWID_LIST,
IN_ROWID_MATCH_RULE, IN_COL_LIST, IN_VAL_LIST, IN_INTERACTION_ID, IN_USER_NAME,
OUT_TMP_TABLE_LIST, OUR_ERROR_MESSAGE, OUT_RETURN_CODE, IN_WINNING_CELL_OVERRIDE);
   DBMS_OUTPUT.PUT_LINE ('OUT_ERROR_MESSAGE = ' || OUT_ERROR_MESSAGE);
   DBMS_OUTPUT.PUT_LINE ('OUT_RETURN_CODE = ' || TO_CHAR(OUT_RETURN_CODE));
   IF OUT_RETURN_CODE < 0 THEN
      ROLLBACK;
   ELSE
      COMMIT;
   END IF;
   CMXUT.DROP_TABLE_IN_LIST(OUT_TMP_TABLE_LIST, OUT_ERROR_MESSAGE,OUT_RETURN_CODE); --Clean up
temporary tables
END;
```

Promote Jobs

For state-enabled objects, a promote job reads the PROMOTE_IND column from an XREF table and for all rows where the column's value is 1, changes the ACTIVE state to on.

Informatica MDM Hub resets PROMOTE_IND after the Promote job has run.

Note: The PROMOTE_IND column on a record is not changed to 0 during the Promote batch process if the record is not promoted.

Stored Procedure Definition for Promote Jobs

```
PROCEDURE CMXSM.AUTO_PROMOTE(
 IN_ROWID_TABLE             IN CHAR(14)
,IN_USER_NAME               IN VARCHAR2(50)
,OUT_ERROR_MESSAGE          OUT VARCHAR2(1024)
,OUT_RETURN_CODE            OUT INT
,IN_ROWID_JOB_GRP_CTRL      IN CHAR(14) DEFAULT NULL
,IN_ROWID_JOB_GRP_ITEM      IN CHAR(14) DEFAULT NULL
)
```

Recalculate Base Object Jobs

Recalculates the best version of the truth for all base objects if you do not use the ROWID_OBJECT_TABLE parameter to identify base objects to recalculate.

Run the Recalculate Base Object job after you change trust settings or validation settings. The MDM Hub does not recalculate the best version of the truth after you change the trust setting or validation setting even if it synchronizes metadata. If you do not recalculate the best version of the truth after you change the trust settings or validation settings, the best version of the truth might be outdated.

You can run the Recalculate Base Object job with or without the ROWID_OBJECT_TABLE parameter. If you run the job with the ROWID_OBJECT_TABLE parameter, the MDM Hub recalculates the best version of the truth for all base objects identified by the ROWID_OBJECT column in the table/inline view. Brackets are required around inline view. If you run the job without the ROWID_OBJECT_TABLE parameter, the MDM Hub recalculates the best version of the truth for all records in the base object. The MDM Hub recalculates the records in batch sizes of MATCH_BATCH_SIZE or a quarter of the number of the records in the table, whichever is less.

Stored Procedure Definition for Recalculate BO Jobs

```
PROCEDURE CMXBV.RECALCULATE_BO(
 IN_TABLE_NAME              CMXLB.CMX_OBJECT_NAME
,IN_ROWID_OBJECT_TABLE      CMXLB.CMX_OBJECT_NAME
,IN_USER_NAME               CMXLB.CMX_USER_NAME
,IN_LOCK_GROUP_STR          IN OUT CMXLB.CMX_LOCK_GROUP_STR
,OUT_TMP_TABLE_LIST         OUT CMXLB.CMX_BIG_STR
,OUT_ERROR_MESSAGE          OUT CMXLB.CMX_MESSAGE
,OUT_RETURN_CODE            OUT INT
,IN_INTERACTION_ID          IN INT DEFAULT NULL
,IN_ROWID_JOB_CTL           IN CMXLB.CMX_ROWID DEFAULT NULL
)
```

Sample Job Execution Script for Recalculate BO Jobs

```
DECLARE
    OUT_ERROR_MESSAGE       VARCHAR2( 1024 );
    OUT_RETURN_CODE         NUMBER;
    OUT_TMP_TABLE_LIST      VARCHAR2(32000);
    IN_LOCK_GROUP_STR       VARCHAR2(100) := NULL;
BEGIN
    BEGIN
       EXECUTE IMMEDIATE 'DROP TABLE TEST_RECALC_BO';
    EXCEPTION WHEN OTHERS THEN
       DBMS_OUTPUT.PUT_LINE('SQLERRM = '|| SQLERRM);
    END;
    EXECUTE IMMEDIATE 'CREATE TABLE TEST_RECALC_BO AS SELECT ROWID_OBJECT FROM C_CONSUMER';'
    CMXBV.RECALCULATE_BO ('C_CUSTOMER', 'TEST_RECALC_BO', 'TNEFF', IN_LOCK_GROUP_STR,
OUT_TMP_TABLE_LIST, OUT_ERROR_MESSAGE, OUT_RETURN_CODE);
    DBMS_OUTPUT.PUT_LINE ('RETURN CODE = ' || OUT_RETURN_CODE);
    DBMS_OUTPUT.PUT_LINE ('MESSAGE IS = ' || OUT_ERROR_MESSAGE);
    IF OUT_RETURN_CODE < 0 THEN
       ROLLBACK;
    ELSE
       COMMIT;
    END IF;
    CMXUT.DROP_TABLE_IN_LIST(OUT_TMP_TABLE_LIST, OUT_ERROR_MESSAGE, OUT_RETURN_CODE); --Clean
up temporary tables
END;
```

Recalculate BVT Jobs

Recalculates the best version of the truth for the specified ROWID_OBJECT.

Run the Recalculate BVT job after you change trust settings or validation settings. The MDM Hub does not recalculate the best version of the truth after you change the trust or validation settings even if it synchronizes metadata. If you do not recalculate the base object after you change the trust settings or validation settings, the best version of the truth might be outdated.

Stored Procedure Definition for Recalculate BVT Jobs

```
PROCEDURE CMXBV.RECALCULATE_BVT(
 IN_TABLE_NAME              CMXLB.CMX_OBJECT_NAME
,IN_ROWID_OBJECT            CMXLB.CMX_ROWID
,IN_USER_NAME               CMXLB.CMX_USER_NAME
,IN_LOCK_GROUP_STR          IN OUT CMXLB.CMX_LOCK_GROUP_STR
```

```
,OUT_TMP_TABLE_LIST          OUT CMXLB.CMX_BIG_STR
,OUT_ERROR_MESSAGE           OUT CMXLB.CMX_MESSAGE
,OUT_RETURN_CODE             OUT INT
,IN_INTERACTION_ID           IN INT DEFAULT NULL
,IN_ROWID_JOB_CTL            IN CMXLB.CMX_ROWID DEFAULT NULL
)
```

Reset Batch Group Status Jobs

Rest Batch Group Status jobs (CMXBG.RESET_BATCHGROUP) resets a batch group.

Note: There are two other related batch group stored procedures:

- Execute Batch Group Jobs (CMXBG.EXECUTE_BATCHGROUP)
- Get Batch Group Status Jobs (CMXBG.GET_BATCHGROUP_STATUS)

Reset Links Jobs

Updates the records in the _LINK table to account for changes in the data. Used with link-style base objects only.

Reset Match Table Jobs

The Reset Match Table job is created automatically after you run a match job and the following conditions exist: if records have been updated to CONSOLIDATION_IND=2, and if you then change your match rules.

Note: This job cannot be run from the Batch Viewer.

Stored Procedure Definition for Reset Match Table Jobs

```
PROCEDURE CMXMA.RESET_MATCH(
 IN_ROWID_TABLE              IN CHAR(14)
,IN_USER_NAME                IN VARCHAR2(50)
,OUT_ERROR_MSG               OUT VARCHAR2(1024)
,RC                          OUT INT
,IN_JOB_GRP_CTRL             IN CHAR(14) DEFAULT NULL
,IN_JOB_GRP_ITEM             IN CHAR(14) DEFAULT NULL
)
```

Sample Job Execution Script for Reset Match Table Jobs

```
DECLARE
    V_ROWID_TABLE            CHAR(14);
    OUT_ERROR_MESSAGE        VARCHAR2(1024);
    OUT_RETURN_CODE          INTEGER;
BEGIN
    SELECT ROWID_TABLE
    INTO   V_ROWID_TABLE
    FROM   C_REPOS_TABLE
    WHERE  TABLE_NAME = 'C_CUSTOMER';
    CMXMA.RESET_MATCH (V_ROWID_TABLE, 'ADMIN', OUT_ERROR_MESSAGE, OUT_RETURN_CODE);
    DBMS_OUTPUT.PUT_LINE ('RETURN MESSAGE: ' || SUBSTR( OUT_ERROR_MESSAGE, 1, 255));
    DBMS_OUTPUT.PUT_LINE ('RETURN CODE: ' || OUT_RETURN_CODE );
    DBMS_OUTPUT.PUT_LINE ('OUT_ERROR_MESSAGE = ' || OUT_ERROR_MESSAGE);
    DBMS_OUTPUT.PUT_LINE ('RC = ' || TO_CHAR(OUT_RETURN_CODE));
    IF OUT_RETURN_CODE < 0 THEN
       ROLLBACK;
    ELSE
       COMMIT;
    END IF;
END;
```

Revalidate Jobs

Revalidate jobs execute the validation logic/rules for records that have been modified since the initial validation during the Load Process.

You can run Revalidate if/when records change post the initial Load process's validation step. If no records change, no records are updated. If some records have changed and get caught by the existing validation rules, the metrics will show the results. Revalidate is executed manually using the batch viewer for base objects.

Note: Revalidate can only be run after an initial load and prior to merge on base objects that have validate rules setup.

Stored Procedure Definition for Revalidate Jobs

```
PROCEDURE CMXUT.REVALIDATE_BO(
 IN_TABLE_NAME              CMXLB.CMX_TABLE_NAME
,IN_USER_NAME               CMXLB.CMX_USER_NAME
,OUT_ERROR_MSG              OUT CMXLB.CMX_MESSAGE
,RC                         OUT INT
,IN_ROWID_JOB_GRP_CTRL      CMXLB.CMX_ROWID := NULL;
,IN_ROWID_JOB_GRP_ITEM      CMXLB.CMX_ROWID := NULL;
,IN_ONLY_CM_DIRTY_IND       INT DEFAULT 0
,IN_RECALC_BVT_IND          INT DEFAULT 0
)
```

Sample Job Execution Script for Revalidate Jobs

```
DECLARE
    IN_TABLE_NAME           VARCHAR2(30);
    IN_USER_NAME            VARCHAR2(30);
    OUT_ERROR_MESSAGE       VARCHAR2(1024);
    RC                      NUMBER;
BEGIN
    IN_TABLE_NAME           := UPPER('&TBL');
    OUT_ERROR_MESSAGE       := NULL;
    RC                      := NULL;
    CMXUT.REVALIDATE_BO (IN_TABLE_NAME, IN_USER_NAME, OUT_ERROR_MESSAGE, RC);
    DBMS_OUTPUT.PUT_LINE ( 'OUT_ERROR_MESSAGE= ' || SUBSTR(OUT_ERROR_MESSAGE,1,200));
    DBMS_OUTPUT.PUT_LINE ('OUT_ERROR_MESSAGE = ' || OUT_ERROR_MESSAGE);
    DBMS_OUTPUT.PUT_LINE ('RC = ' || TO_CHAR(RC));
    IF RC < 0 THEN
       ROLLBACK;
    ELSE
       COMMIT;
    END IF;
END;
```

Stage Jobs

Stage jobs copy records from a landing to a staging table.

During execution, Stage jobs optionally cleanse data according to the current cleanse settings.

Identifiers for Executing Stage Jobs

Identifiers are used to execute the stored procedure associated with this batch job.

Dependencies for Stage Jobs

Each Stage job is dependent on the successful completion of the Extraction Transform Load (ETL) process responsible for loading the Landing table used by the Stage job. There are no dependencies between Stage jobs.

Successful Completion of Stage Jobs

A Stage job must complete with a RUN_STATUS of 0 (Completed Successfully) or 1 (Completed with Errors) to be considered successful. On successful completion of a Stage job, the Load job for the target staging table can be run, provided that all other dependencies for the Load job have been met.

Stored Procedure Definition for Stage Jobs

```
PROCEDURE CMXCL.START_CLEANSE(
 IN_ROWID_TABLE_OBJECT      IN VARCHAR2(500) --From the view
,IN_USER_NAME               IN VARCHAR2(50)
,OUT_ERROR_MSG              OUT VARCHAR2(1024)
,OUT_ERROR_CODE             OUT INT
,IN_STG_ROWID_TABLE         IN VARCHAR2(500) --rowid_table_object
,IN_RUN_SYNCH               IN VARCHAR2(500) --Set to true, else runs asynch
,IN_ROWID_JOB_GRP_CTRL      IN CHAR(14) DEFAULT NULL
,IN_ROWID_JOB_GRP_ITEM      IN CHAR(14) DEFAULT NULL
)
```

Sample Job Execution Script for Stage Jobs

```
DECLARE
    IN_ROWID_TABLE_OBJECT   VARCHAR2(500);
    IN_USER_NAME            VARCHAR2(50);
    OUT_ERROR_MSG           VARCHAR2(1024);
    OUT_ERROR_CODE          NUMBER;
    IN_STG_ROWID_TABLE      VARCHAR2(500);
    IN_RUN_SYNCH            VARCHAR2(500);

BEGIN
    IN_STG_ROWID_TABLE      := NULL;
    IN_ROWID_TABLE_OBJECT   := NULL;
    IN_RUN_SYNCH            := NULL;
    IN_USER_NAME            := NULL;
    OUT_ERROR_MSG           := NULL;
    OUT_ERROR_CODE          := NULL;
    SELECT A.ROWID_TABLE, A.ROWID_TABLE_OBJECT
    INTO IN_STG_ROWID_TABLE, IN_ROWID_TABLE_OBJECT
    FROM C_REPOS_TABLE_OBJECT_V A, C_REPOS_TABLE B
    WHERE A.OBJECT_NAME = 'CMXCL.START_CLEANSE'
    AND B.ROWID_TABLE = A.ROWID_TABLE
    AND B.TABLE_NAME = 'C_HMO_ADDRESS'
    AND A.VALID_IND = 1;
    CMXCL.START_CLEANSE (IN_ROWID_TABLE_OBJECT, IN_USER_NAME, OUT_ERROR_MSG, OUT_ERROR_CODE,
IN_STG_ROWID_TABLE, IN_RUN_SYNCH, IN_ROWID_JOB_GRP_CTRL, IN_ROWID_JOB_GRP_ITEM);
    DBMS_OUTPUT.PUT_LINE(' MESSAGE IS = ' || OUT_ERROR_MSG);
    DBMS_OUTPUT.PUT_LINE('OUT_ERROR_MESSAGE = ' || OUT_ERROR_MSG);
    DBMS_OUTPUT.PUT_LINE('OUT_ERROR_CODE = ' || TO_CHAR(OUT_ERROR_CODE));
    IF OUT_ERROR_CODE < 0 THEN
       ROLLBACK;
    ELSE
       COMMIT;
    END IF;
END;
```

Synchronize Jobs

You must run the Synchronize job after any changes are made to the schema trust settings.

The Synchronize job is created when any changes are made to the schema trust settings.

Running Synchronize Jobs

To run the Synchronize job, navigate to the Batch Viewer, find the correct Synchronize job for the base object, and run it.

Informatica MDM Hub updates the metadata for the base objects that have trust enabled after initial load has occurred.

Stored Procedure Definition for Synchronize Jobs

```
PROCEDURE CMXUT.SYNC(
 IN_ROWID_TABLE             CMXLB.CMX_ROWID
,IN_USER_NAME               CMXLB.CMX_USER_NAME
,OUT_ERROR_MSG              OUT CMXLB.CMX_MESSAGE
,OUT_RETURN_CODE            OUT INT
,IN_JOB_GRP_CTRL            CMXLB.CMX_ROWID DEFAULT NULL
,IN_JOB_GRP_ITEM            CMXLB.CMX_ROWID DEFAULT NULL
,IN_ONLY_CM_DIRTY_IND       INT DEFAULT 0
)
```

Sample Job Execution Script for Synchronize Jobs

```
DECLARE
    V_ROWID_TABLE          CHAR(14);
    OUT_ERROR_MESSAGE      VARCHAR2(1024);
    OUT_RETURN_CODE        INTEGER;
BEGIN
    SELECT ROWID_TABLE
    INTO   V_ROWID_TABLE
    FROM   C_REPOS_TABLE
    WHERE  TABLE_NAME = 'C_CUSTOMER';
    CMXUT.SYNCH (V_ROWID_TABLE, 'ADMIN', OUT_ERROR_MESSAGE, OUT_RETURN_CODE);
    DBMS_OUTPUT.PUT_LINE ('RETURN MESSAGE: ' || SUBSTR(OUT_ERROR_MESSAGE, 1, 255));
    DBMS_OUTPUT.PUT_LINE ('RETURN CODE: ' || OUT_RETURN_CODE);
    IF OUT_RETURN_CODE < 0 THEN
       ROLLBACK;
    ELSE
       COMMIT;
    END IF;
END;
```

Executing Batch Groups Using Stored Procedures

You can use stored procedures to execute batch groups for the Informatica MDM Hub implementation.

About Running Batch Groups

A *batch group* is a collection of individual batch jobs such as Stage, Load, and Match jobs that can be run with a single command. The batch groups can be run sequentially or in parallel according to the configuration.

When one job has an error, the group will stop, and no more jobs are started. However, jobs that are running are completed.

You can run batch groups using stored procedures and job scheduling software such as Tivoli, and CA Unicenter. Informatica MDM Hub provides stored procedures for managing batch groups.

You can also use the Batch Group tool in the Hub Console to configure and run batch groups. However, to schedule batch groups, you need to use stored procedures.

Note: If a batch group fails and you do not click either the **Set to Restart** button or the **Set to Incomplete** button in the Logs for My Batch Group list, Informatica MDM Hub restarts the batch job from the prior failed level.

Stored Procedures for Batch Groups

Informatica MDM Hub provides the following stored procedures for managing batch groups:

Stored Procedure	Description
CMXBG.EXECUTE_BATCHGROUP	Performs an HTTP POST to the SIF ExecuteBatchGroupRequest.
CMXBG.RESET_BATCHGROUP	Performs an HTTP POST to the SIF ResetBatchGroupRequest.
CMXBG.GET_BATCHGROUP_STATUS	Performs an HTTP POST to the SIF GetBatchGroupStatusRequest.

In addition to using parameters that are associated with the corresponding SIF request, these stored procedures require the following parameters:

- URL of the Hub Server (for example, http://localhost:7001/cmx/request)
- User name and password
- Target Operational Reference Store

Note: The stored procedures construct an XML message, perform an HTTP POST to a server URL using SIF, and return the results.

CMXBG.EXECUTE_BATCHGROUP

Execute Batch Group jobs execute a batch group. Execute Batch Groups jobs have an option to execute asynchronously, but not to receive a JMS response for asynchronous execution. If you need to use asynchronous execution and need to know when execution is finished, then poll with the cmxbg.get_batchgroup_status stored procedure. Alternatively, if you need to receive a JMS response for asynchronous execution, then execute the batch group directly in an external application (instead of a job execution script) by invoking the SIF ExecuteBatchGroup request.

Signature

```
FUNCTION CMXBG.EXECUTE_BATCHGROUP(
  IN_MRM_SERVER_URL         IN VARCHAR2(500)
, IN_USERNAME               IN VARCHAR2(500)
, IN_PASSWORD               IN VARCHAR2(500)
, IN_ORSID                  IN VARCHAR2(500)
, IN_BATCHGROUP_UID         IN VARCHAR2(500)
, IN_RESUME                 IN VARCHAR2(500)
, IN_ASYNCRONOUS            IN VARCHAR2(500)
, OUT_ROWID_BATCHGROUP_LOG  OUT VARCHAR2(500)
, OUT_ERROR_MSG             OUT VARCHAR2(500)
) RETURN NUMBER --Return the error code
```

Parameters

Name	Description
IN_MRM_SERVER_URL	Hub Server SIF URL.
IN_USERNAME	User account with role-based permissions to execute batch groups.
IN_PASSWORD	Password for the user account with role-based permissions to execute batch groups.
IN_ORSID	ORS ID as shown in **Console** > **Configuration** > **Databases**.

Name	Description
IN_BATCHGROUP_UID	Informatica MDM Hub Object UID of batch group to [execute, reset, get status, etc.].
IN_RESUME	One of the following values: true: if previous execution failed, resume at that point false: regardless of previous execution, start from the beginning
IN_ASYNCRONOUS	Specifies whether to execute asynchronously or synchronously. One of the following values: true: start execution and return immediately (asynchronous execution). false: return when group execution is complete (synchronous execution).

Returns

Parameter	Description
OUT_ROWID_BATCHGROUP_LOG	c_repos_job_group_control.rowid_job_group_control
OUT_ERROR_MSG	Error message text.
NUMBER	Error code. If zero (0), then the stored procedure completed successfully. If one (1), then the stored procedure returns an explanation in out_error_msg.

Sample Job Execution Script for Execute Batch Group Jobs

```
DECLARE
   OUT_ROWID_BATCHGROUP_LOG      CMXLB.CMX_SMALL_STR;
   OUT_ERROR_MSG                 CMXLB.CMX_SMALL_STR;
   RET_VAL                       INT;
BEGIN
   RET_VAL := CMXBG.EXECUTE_BATCHGROUP(
            'HTTP://LOCALHOST:7001/CMX/REQUEST/PROCESS/'
            , 'ADMIN'
            , 'ADMIN'
            , 'LOCALHOST-MRM-XU_3009'
            , 'BATCH_GROUP.MYBATCHGROUP'
            , 'true' -- OR 'false'
            , 'true' -- OR 'false'
            , OUT_ROWID_BATCHGROUP_LOG
            , OUT_ERROR_MSG
      );
   CMXLB.DEBUG_PRINT('EXECUTE_BATCHGROUP:    ' || ' CODE='|| RET_VAL || ' MESSAGE='||
OUT_ERROR_MSG ||    ' | OUT_ROWID_BATCHGROUP_LOG='|| OUT_ROWID_BATCHGROUP_LOG);
   );
   COMMIT;
END;
```

CMXBG.RESET_BATCHGROUP

Reset Batch Group Status jobs resets a batch group.

Note: In addition to this stored procedure, there are Java API requests and the SOAP and HTTP XML protocols available using Services Integration Framework (SIF). The Reset Batch Group Status job has the following SIF API requests available: ResetBatchGroup.

Signature

```
FUNCTION CMXBG.RESET_BATCHGROUP(
  IN_MRM_SERVER_URL         IN VARCHAR2(500)
```

```
, IN_USERNAME                IN VARCHAR2(500)
, IN_PASSWORD                IN VARCHAR2(500)
, IN_ORSID                   IN VARCHAR2(500)
, IN_BATCHGROUP_UID          IN VARCHAR2(500)
, OUT_ROWID_BATCHGROUP_LOG   OUT VARCHAR2(500)
, OUT_ERROR_MSG              OUT VARCHAR2(500)
) RETURN NUMBER --Return the error code
```

Parameters

Name	Description
IN_MRM_SERVER_URL	Hub Server SIF URL.
IN_USERNAME	User account with role-based permissions to execute batch groups.
IN_PASSWORD	Password for the user account with role-based permissions to execute batch groups.
IN_ORSID	ORS ID as specified in the Database tool in the Hub Console.
IN_BATCHGROUP_UID	Informatica MDM Hub Object UID of batch group to [execute, reset, get status of, and so on].

Returns

Parameter	Description
OUT_ROWID_BATCHGROUP_LOG	c_repos_job_group_control.rowid_job_group_control
OUT_ERROR_MSG	Error message text.
NUMBER	Error code. If zero (0), then the stored procedure completed successfully. If one (1), then the stored procedure returns an explanation in out_error_msg.

Sample Job Execution Script for Reset Batch Group Jobs

```
DECLARE
   OUT_ROWID_BATCHGROUP_LOG      CMXLB.CMX_SMALL_STR;
   OUT_ERROR_MSG                 CMXLB.CMX_SMALL_STR;
   RET_VAL                       INT;
BEGIN
   RET_VAL := CMXBG.RESET_BATCHGROUP(
      'HTTP://LOCALHOST:7001/CMX/REQUEST/PROCESS/'
      , 'ADMIN'
      , 'ADMIN'
      ,'LOCALHOST-MRM-XU_3009'
      , 'BATCH_GROUP.MYBATCHGROUP'
      , OUT_ROWID_BATCHGROUP_LOG
      , OUT_ERROR_MSG
   );
   CMXLB.DEBUG_PRINT('RESET_BATCHGROUP: CODE=' || RET_VAL || ' MESSAGE=' || OUT_ERROR_MSG || '
OUT_ROWID_BATCHGROUP_LOG=' || OUT_ROWID_BATCHGROUP_LOG);
/
```

CMXBG.GET_BATCHGROUP_STATUS

Get Batch Group Status jobs return the batch group status.

Note: In addition to this stored procedure, there are Java API requests and the SOAP and HTTP XML protocols available using Services Integration Framework (SIF). The Get Batch Group Status job has the following SIF API requests available: GetBatchGroupStatus.

Signature

```
FUNCTION CMXBG.GET_BATCHGROUP_STATUS(
  IN_MRM_SERVER_URL          IN VARCHAR2(500)
, IN_USERNAME                IN VARCHAR2(500)
, IN_PASSWORD                IN VARCHAR2(500)
, IN_ORSID                   IN VARCHAR2(500)
, IN_BATCHGROUP_UID          IN VARCHAR2(500)
, IN_ROWID_BATCHGROUP_LOG    IN VARCHAR2(500)
, OUT_ROWID_BATCHGROUP       OUT VARCHAR2(500)
, OUT_ROWID_BATCHGROUP_LOG   OUT VARCHAR2(500)
, OUT_START_RUNDATE          OUT VARCHAR2(500)
, OUT_END_RUNDATE            OUT VARCHAR2(500)
, OUT_RUN_STATUS             OUT VARCHAR2(500)
, OUT_STATUS_MESSAGE         OUT VARCHAR2(500)
, OUT_ERROR_MSG              OUT VARCHAR2(500)
) RETURN NUMBER --Return the error code
```

Parameters

Name	Description
IN_MRM_SERVER_URL	Hub Server SIF URL.
IN_USERNAME	User account with role-based permissions to execute batch groups.
IN_PASSWORD	Password for the user account with role-based permissions to execute batch groups.
IN_ORSID	ORS ID as specified in the Database tool in the Hub Console.
IN_BATCHGROUP_UID	Informatica MDM Hub Object UID of batch group to [execute, reset, get status of, and so on]. If IN_ROWID_BATCHGROUP_LOG is null, the most recent log for this group will be used.
IN_ROWID_BATCHGROUP_LOG	c_repos_job_group_control.rowid_job_group_control Either IN_BATCHGROUP_UID or IN_ROWID_BATCHGROUP_LOG is required.

Returns

Parameter	Description
OUT_ROWID_BATCHGROUP	c_repos_job_group.rowid_job_group
OUT_ROWID_BATCHGROUP_LOG	c_repos_job_group_control.rowid_job_group_control
OUT_START_RUNDATE	Date / time when this batch job started.
OUT_END_RUNDATE	Date / time when this batch job ended.
OUT_RUN_STATUS	Job execution status code that is displayed in the Batch Group tool.
OUT_STATUS_MESSAGE	Job execution status message that is displayed in the Batch Group tool.
OUT_ERROR_MSG	Error message text for this stored procedure call, if applicable.
NUMBER	Error code. If zero (0), then the stored procedure completed successfully. If one (1), then the stored procedure returns an explanation in out_error_msg.

Sample Job Execution Script for Get Batch Group Status Jobs

```
DECLARE
   OUT_ROWID_BATCHGROUP          CMXLB.CMX_SMALL_STR;
   OUT_ROWID_BATCHGROUP_LOG      CMXLB.CMX_SMALL_STR;
   OUT_START_RUNDATE             CMXLB.CMX_SMALL_STR;
   OUT_END_RUNDATE               CMXLB.CMX_SMALL_STR;
   OUT_RUN_STATUS                CMXLB.CMX_SMALL_STR;
   OUT_STATUS_MESSAGE            CMXLB.CMX_SMALL_STR;
   OUT_ERROR_MSG                 CMXLB.CMX_SMALL_STR;
   OUT_RETURNCODE                INT;
   RET_VAL                       INT;
BEGIN
   RET_VAL := CMXBG.GET_BATCHGROUP_STATUS(
      'HTTP://LOCALHOST:7001/CMX/REQUEST/PROCESS/'
      , 'ADMIN'
      , 'ADMIN'
      ,'LOCALHOST-MRM-XU_3009'
      , 'BATCH_GROUP.MYBATCHGROUP'
      , NULL
      , OUT_ROWID_BATCHGROUP
      , OUT_ROWID_BATCHGROUP_LOG
      , OUT_START_RUNDATE
      , OUT_END_RUNDATE
      , OUT_RUN_STATUS
      , OUT_STATUS_MESSAGE
      , OUT_ERROR_MSG
   );
   CMXLB.DEBUG_PRINT('GET_BATCHGROUP_STATUS: CODE='|| RET_VAL || ' MESSAGE='|| OUT_ERROR_MSG ||
' STATUS=' || OUT_STATUS_MESSAGE || ' | OUT_ROWID_BATCHGROUP_LOG='|| OUT_ROWID_BATCHGROUP_LOG);
END;
/
```

Developing Custom Stored Procedures for Batch Jobs

You can create and register custom stored procedures for batch jobs that can be added to batch groups for the Informatica MDM Hub implementation.

About Custom Stored Procedures

Informatica MDM Hub allows you to create and run custom stored procedures for batch jobs.

After developing the custom stored procedure, you must register it in order to make it available to users as batch jobs in the Batch Viewer and Batch Group tools in the Hub Console.

Required Execution Parameters for Custom Batch Jobs

The following parameters are required for custom batch jobs. During its execution, a custom batch job can call cmxut.set_metric_value to register metrics.

Signature

```
PROCEDURE EXAMPLE_JOB(
 IN_ROWID_TABLE_OBJECT      IN CHAR(14) --C_REPOS_TABLE_OBJECT.ROWID_TABLE_OBJECT, RESULT OF
CMXUT.REGISTER_CUSTOM_TABLE_OBJECT
,IN_USER_NAME               IN VARCHAR2(50) --Username calling the function
,IN_ROWID_JOB               IN CHAR(14) --C_REPOS_JOB_CONTROL.ROWID_JOB, for reference, do not
update status
,OUT_ERR_MSG                OUT VARCHAR --Message about success or error
,OUT_ERR_CODE               OUT INT -- >=0: Completed successfully. <0: Error
)
```

Parameters

Name	Description
in_rowid_table_object IN cmxlb.cmx_rowid	c_repos_table_object.rowid_table_object Result of cmxut.REGISTER_CUSTOM_TABLE_OBJECT
in_user_name IN cmxlb.cmx_user_name	User name calling the function.

Returns

Parameter	Description
out_err_msg	Error message text.
out_err_code	Error code.

Registering a Custom Stored Procedure

You must register a custom stored procedure with Informatica MDM Hub in order to make it available to users in the Batch Group tool in the Hub Console. You can register the same custom job multiple times for different tables (in_rowid_table). To register a custom stored procedure, you need to call this stored procedure in c_repos_table_object:

```
CMXUT.REGISTER_CUSTOM_TABLE_OBJECT
```

Signature

```
PROCEDURE REGISTER_CUSTOM_TABLE_OBJECT(
  IN_ROWID_TABLE           IN CHAR(14)
, IN_OBJ_FUNC_TYPE_CODE    IN VARCHAR
, IN_OBJ_FUNC_TYPE_DESC    IN VARCHAR
, IN_OBJECT_NAME           IN VARCHAR
)
```

Parameters

Name	Description
IN_ROWID_TABLE CMXLB.CMX_ROWID	Foreign key to c_repos_table.rowid_table. When the Hub Server calls the custom job in a batch group, this value is passed in.
IN_OBJ_FUNC_TYPE_CODE	Job type code. Must be 'A' for batch group custom jobs.
IN_OBJ_FUNC_TYPE_DESC	Display name for the custom batch job in the Batch Groups tool in the Hub Console.
IN_OBJECT_NAME	package.procedure name of the custom job.

Example

```
BEGIN
  cmxut.REGISTER_CUSTOM_TABLE_OBJECT (
    'SVR1.RS1B    '     -- c_repos_table.rowid_table
    ,'A'                -- Job type, must be 'A' for batch group
    ,'CMXBG_EXAMPLE.UPDATE_TABLE EXAMPLE'  -- Display name
    ,'CMXBG_EXAMPLE.UPDATE_TABLE'          -- Package.procedure
  );
END;
```

Removing Data from a Base Object and Supporting Metadata Tables

Use the CMXUT.CLEAN_TABLE procedure to remove all data from a base object and its supporting metadata tables. If a base object is referenced by a foreign key in another base object, then the referencing base object must be empty before you run cmxut.clean_table for the referenced base object.

Example

```
DECLARE
   IN_TABLE_NAME            VARCHAR2(30);
   OUT_ERROR_MESSAGE        VARCHAR2(1024);
   RC                       NUMBER;
BEGIN
   IN_TABLE_NAME            := 'C_BO_TO_CLEAN'; --Name of the BO table
   OUT_ERROR_MESSAGE        := NULL;        --Return msg; output parameter
   RC                       := NULL;        --Return code; output parameter
   CMXUT.CLEAN_TABLE (IN_TABLE_NAME, OUT_ERROR_MESSAGE, RC);
   COMMIT;
END;
```

Writing Messages to the Informatica MDM Hub Database Debug Log

Use the CMXLB.DEBUG_PRINT procedure to write your own messages to Informatica MDM Hub database debug log file.

The message is written to the log if logging is enabled and if it has been configured correctly.

Example

```
DECLARE
   IN_DEBUG_TEXT            VARCHAR2(32000);
BEGIN
   IN_DEBUG_TEXT            := NULL; --String that you want to print in the log file
   CMXUT.DEBUG_PRINT(IN_DEBUG_TEXT);
   COMMIT;
 END;
```

Example Custom Stored Procedure

```
CREATE OR REPLACE PACKAGE CMXBG_EXAMPLE
AS
PROCEDURE UPDATE_TABLE(
     IN_ROWID_TABLE_OBJECT  IN  CMXLB.CMX_ROWID
    ,IN_USER_NAME           IN  CMXLB.CMX_USER_NAME
    ,IN_ROWID_JOB           IN  CMXLB.CMX_ROWID
    ,OUT_ERR_MSG            OUT VARCHAR
    ,OUT_ERR_CODE           OUT INT
);
END CMXBG_EXAMPLE;
/
CREATE OR REPLACE PACKAGE BODY CMXBG_EXAMPLE
AS
PROCEDURE UPDATE_TABLE(
            IN_ROWID_TABLE_OBJECT IN CMXLB.CMX_ROWID
            ,IN_USER_NAME IN CMXLB.CMX_USER_NAME
            ,IN_ROWID_JOB IN CMXLB.CMX_ROWID
            ,OUT_ERR_MSG OUT VARCHAR
            ,OUT_ERR_CODE OUT INT ) is

   BEGIN
      DECLARE
         CUTOFF_DATE        DATE;
         RECORD_COUNT       INT;
         RUN_STATUS         INT;
         STATUS_MESSAGE     VARCHAR2 (2000);
         START_DATE         DATE   := SYSDATE;
         MRM_ROWID_TABLE    CMXLB.CMX_ROWID;
```

```
      OBJ_FUNC_TYPE      CHAR (1);
      JOB_ID             CHAR (14);
      SQL_STMT           VARCHAR2 (2000);
      TABLE_NAME         VARCHAR2(30);
      RET_CODE           INT;
      REGISTER_JOB_ERR   EXCEPTION;
   BEGIN
      SQL_STMT :=
            'ALTER SESSION SET NLS_DATE_FORMAT=''DD MON YYYY HH24:MI:SS''';

      EXECUTE IMMEDIATE SQL_STMT;
      CMXLB.DEBUG_PRINT ('START OF CUSTOM BATCH JOB...');
      OBJ_FUNC_TYPE := 'A';

      SELECT ROWID_TABLE
        INTO MRM_ROWID_TABLE
        FROM C_REPOS_TABLE_OBJECT
       WHERE ROWID_TABLE_OBJECT = IN_ROWID_TABLE_OBJECT;

      SELECT START_RUN_DATE
        INTO CUTOFF_DATE
        FROM C_REPOS_JOB_CONTROL
       WHERE ROWID_JOB = IN_ROWID_JOB;
      IF CUTOFF_DATE IS NULL THEN
         CUTOFF_DATE := SYSDATE - 7;
      END IF;

      SELECT TABLE_NAME
        INTO TABLE_NAME
        FROM C_REPOS_TABLE RT, C_REPOS_TABLE_OBJECT RTO
      WHERE RTO.ROWID_TABLE_OBJECT = IN_ROWID_TABLE_OBJECT
        AND RTO.ROWID_TABLE = RT.ROWID_TABLE;

      -- THE REAL WORK!
      SQL_STMT :=
            'UPDATE ' || TABLE_NAME || ' SET ZIP4 = ''0000'', LAST_UPDATE_DATE = '''
         || CUTOFF_DATE
         || ''''
         || ' WHERE ZIP4 IS NULL';
      CMXLB.DEBUG_PRINT (SQL_STMT);
      EXECUTE IMMEDIATE SQL_STMT;
      RECORD_COUNT := SQL%ROWCOUNT;
      COMMIT;
      -- For testing, sleep to make the procedure take longer
      -- dbms_lock.sleep(5);
      -- Set zero or many metrics about the job
      CMXUT.SET_METRIC_VALUE (IN_ROWID_JOB, 1, RECORD_COUNT,
       OUT_ERR_CODE, OUT_ERR_MSG);
      COMMIT;
      IF RECORD_COUNT <= 0 THEN
         OUT_ERR_MSG := 'FAILED TO UPDATE RECORDS.';
         OUT_ERR_CODE := -1;
      ELSE
          IF OUT_ERR_CODE >= 0 THEN
             OUT_ERR_MSG := 'COMPLETED SUCCESSFULLY.';
          END IF;
          -- Else keep success code and msg from set_metric_value
      END IF;
   EXCEPTION
      WHEN OTHERS
      THEN
         OUT_ERR_CODE := SQLCODE;
         OUT_ERR_MSG := SUBSTR (SQLERRM, 1, 200);
   END;
END;
END CMXBG_EXAMPLE;
/
```

Author

Brintha Bennet